

Laporan Tugas Besar

Arion Compiler

IF2224 Teori Bahasa Formal dan Otomata



Oleh :
khusus 1352____

Daniel Anindito Nugroho 13524002

Ishak Palentino Napitupulu 13524022

Al Farabi 13524086

Ishaq Irfan Farizal 13524094

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

I. Teori Singkat

A. Analisis Sintaksis

Analisis sintaksis atau syntax analysis adalah tahap kompilasi setelah analisis leksikal. Pada tahap ini, token-token yang dihasilkan oleh lexer diperiksa berdasarkan aturan grammar bahasa pemrograman. Jika urutan token sesuai grammar, parser membangun struktur hierarkis berupa parse tree. Jika tidak sesuai, parser melaporkan kesalahan sintaksis beserta lokasi token yang bermasalah.

Pada proyek Arion Compiler, alur masukan adalah sebagai berikut: source .txt -> Lexer -> token list -> Parser -> Parse Tree.

B. Parser

Parser adalah komponen compiler yang menentukan apakah token-token masukan membentuk program yang valid secara sintaksis. Parser bekerja dengan mencocokkan token terhadap grammar yang sudah ditentukan. Pada proyek ini, parser juga bertugas membentuk parse tree agar struktur program dapat diamati secara eksplisit.

Tugas utama parser pada Milestone 2 adalah:

- Memastikan struktur program mengikuti aturan Arion.
- Memastikan urutan deklarasi benar, yaitu const, lalu type, lalu var, lalu subprogram.
- Membangun parse tree dari semua rule grammar.
- Melaporkan error sintaksis dengan informasi baris, kolom, token aktual, dan token yang diharapkan.

C. Context-Free Grammar

Grammar yang digunakan pada parser Arion adalah context-free grammar. Setiap rule memiliki bentuk produksi yang mendefinisikan bagaimana sebuah non-terminal dapat diturunkan menjadi terminal atau non-terminal lain. Contoh: `<assignment-statement> -> <variable> becomes <expression>`

D. Recursive Descent Parser

Recursive descent parser adalah teknik parsing top-down yang mengimplementasikan setiap non-terminal grammar sebagai satu fungsi. Contohnya, rule `<statement>` diimplementasikan sebagai fungsi `parse_statement()`, sedangkan rule `<if-statement>` diimplementasikan sebagai fungsi `parse_if_statement()`.

Pendekatan ini dipilih karena grammar Milestone 2 relatif langsung dipetakan ke fungsi-fungsi C++. Recursive descent juga memudahkan pembagian tugas antar anggota karena setiap anggota dapat mengerjakan kumpulan rule tertentu pada file yang berbeda.

E. Parse Tree

Parse tree adalah pohon yang menunjukkan proses derivasi program berdasarkan grammar. Berbeda dengan abstract syntax tree (AST), parse tree masih mempertahankan detail grammar, termasuk node wrapper seperti `<statement>`, `<variable>`, `<additive-operator>`, `<relational-operator>`, dan `<multiplicative-operator>`.

F. Error Handling

Parser dirancang agar tidak langsung crash ketika menemukan kesalahan. Error dikumpulkan pada `std::vector<std::string> errors`. Ketika memungkinkan, parser menambahkan node `<error>` dan melakukan sinkronisasi ke token aman seperti semicolon, endsy, atau period.

Minimal pesan error yang dihasilkan mencakup:

- Nomor baris.
- Nomor kolom.
- Token yang ditemukan.
- Token yang diharapkan.

II. Perancangan & Implementasi

A. Arsitektur Umum Parser

Milestone 2 menambahkan tahap syntax analysis pada Arion Compiler. Lexer dari Milestone 1 tetap digunakan sebagai tahap awal untuk mengubah source code menjadi token list. Parser kemudian membaca token list tersebut untuk memvalidasi struktur program berdasarkan grammar Arion dan membangun parse tree.

Parser tidak membaca karakter mentah secara langsung. Semua keputusan parsing dibuat berdasarkan TokenType yang dihasilkan lexer. Dengan demikian, tanggung jawab lexer dan parser tetap terpisah: lexer menangani validitas leksikal, sedangkan parser menangani validitas sintaksis.

Token `TOKEN_COMMENT` dan `TOKEN_NEWLINE` difilter sebelum parsing karena tidak memengaruhi struktur grammar. Sebaliknya, `TOKEN_ERROR` tidak boleh dihapus diam-diam. Jika lexer menghasilkan token error, program melaporkan lexical error dan parsing dihentikan agar parser tidak bekerja pada token list yang sudah rusak secara leksikal.

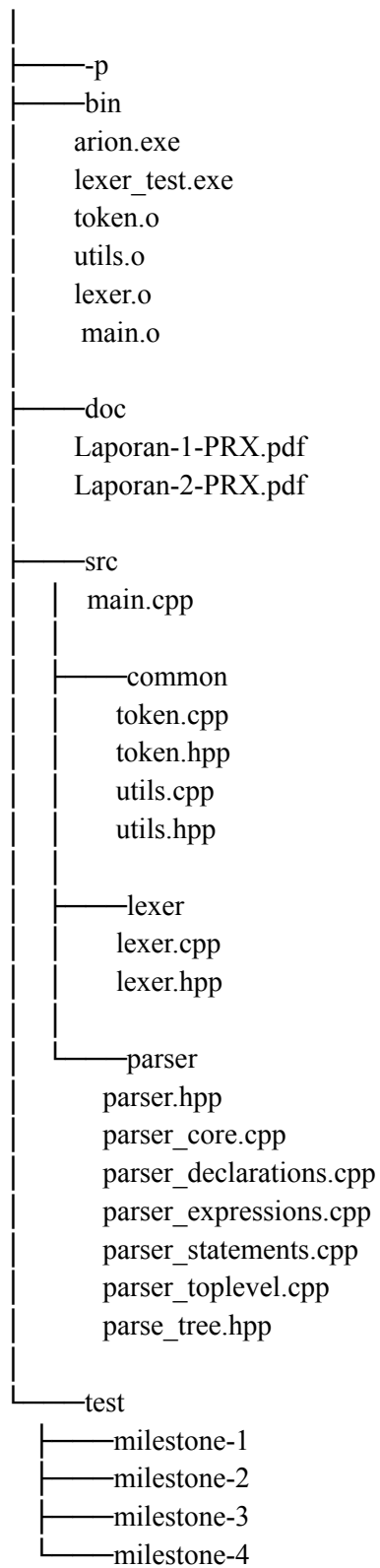
Parser menggunakan recursive descent. Setiap non-terminal grammar direpresentasikan sebagai fungsi C++. Contohnya, rule `<program>` diimplementasikan oleh `parse_program()`, rule `<statement>` oleh `parse_statement()`, dan rule `<expression>` oleh `parse_expression()`. Pendekatan ini cocok untuk proyek karena 43 grammar rule dapat dipetakan secara eksplisit ke fungsi-fungsi parser, sehingga pembagian tugas antar anggota menjadi jelas.

B. Struktur Program

Struktur folder parser pada Milestone 2 adalah:

PRX-Tubes-IF2224-2026/

```
| .gitignore  
| Makefile  
| README.md
```



C. Daftar Lengkap Grammar

No.	Node	Produksi
1	<program>	<program-header> <declaration-part> <compound-statement> period
2	<program-header>	programsy ident semicolon
3	<declaration-part>	<const-declaration>* <type-declaration>* <var-declaration>* <subprogram-declaration>*
4	<const-declaration>	constsy (ident eql <constant> semicolon)+
5	<constant>	charcon string (plus minus)? (ident intcon realcon)
6	<type-declaration>	typesy (ident eql <type> semicolon)+
7	<var-declaration>	varsy (<identifier-list> colon <type> semicolon)+
8	<identifier-list>	ident (comma ident)*
9	<type>	ident <array-type> <range> <enumerated> <record-type>
10	<array-type>	arraysy lbrack (<range> ident) rbrack ofsy <type>
11	<range>	<constant> period period <constant>
12	<enumerated>	lparent ident (comma ident)* rparent
13	<record-type>	recordsy <field-list> endsy
14	<field-list>	<field-part> (semicolon <field-part>)*
15	<field-part>	<identifier-list> colon <type>
16	<subprogram-declaration>	<procedure-declaration> <function-declaration>
17	<procedure-declaration>	proceduresy ident <formal-parameter-list>? semicolon <block> semicolon
18	<function-declaration>	functionsy ident <formal-parameter-list>? colon ident semicolon <block> semicolon
19	<block>	<declaration-part> <compound-statement>
20	<formal-parameter-list>	lparent <parameter-group> (semicolon <parameter-group>)* rparent
21	<parameter-group>	<identifier-list> colon (ident <array-type>)
22	<compound-statement>	beginsy <statement-list> endsy
23	<statement-list>	<statement> (semicolon <statement>)*
24	<statement>	(<assignment-statement> <if-statement> <case-statement> <while-statement> <repeat-statement> <for-statement> <procedure/function-call>)?

25	<variable>	ident <component-variable>*
26	<component-variable>	(lbrack <index-list> rbrack) (period ident)
27	<index-list>	(intcon charcon ident) (comma <index-list>)*
28	<assignment-statement>	<variable> becomes <expression>
29	<if-statement>	ifsy <expression> thensy <statement> (elsesy <statement>)?
30	<case-statement>	casesy <expression> ofsy <case-block> endsy
31	<case-block>	<constant> (comma <constant>)* colon <statement> (semicolon <case-block>?)*
32	<while-statement>	whilesy <expression> dosy <statement>
33	<repeat-statement>	repeatsy <statement-list> untilsy <expression>
34	<for-statement>	forsy ident becomes <expression> (tosy downtosy) <expression> dosy <statement>
35	<procedure/function-call>	ident lparent <parameter-list> rparent
36	<parameter-list>	<expression> (comma <expression>)*
37	<expression>	<simple-expression> (<relational-operator> <simple-expression>)?
38	<simple-expression>	(plus minus)? <term> (<additive-operator> <term>)*
39	<term>	<factor> (<multiplicative-operator> <factor>)*
40	<factor>	ident intcon realcon charcon string (lparent <expression> rparent) (notsy <factor>) <procedure/function-call> <variable>
41	<relational-operator>	eq neq gtr geq lss leq
42	<additive-operator>	plus minus orsy
43	<multiplicative-operator>	times rdiv idiv imod andsy

D. Implementasi Program

1. Desain Parse Tree

Parse tree direpresentasikan oleh kelas ParseTreeNode. Setiap node memiliki nama dan daftar child. Node terminal menyimpan nama token, sedangkan token yang memiliki nilai seperti ident, intcon, realcon, charcon, string, dan unknown ditampilkan bersama lexeme-nya. Parse tree dicetak menggunakan struktur bercabang agar hubungan parent-child mudah dibaca.

parse_tree.hpp

```

class ParseTreeNode {
public:
    std::string name;
    std::vector<std::shared_ptr<ParseTreeNode>> children;

    explicit ParseTreeNode(const std::string& name) : name(name) {}

    void add_child(const std::shared_ptr<ParseTreeNode>& child) {
        if (child) {
            children.push_back(child);
        } else {
            children.push_back(std::make_shared<ParseTreeNode>("<empty>"));
        }
    }

    bool is_terminal() const {
        return children.empty();
    }

    void print_children(std::ostream& os, const std::string& prefix = "") const {
        for (size_t i = 0; i < children.size(); ++i) {
            const bool is_last = i == children.size() - 1;
            os << prefix << (is_last ? "\xe2\x94\x94\xe2\x94\x80\xe2\x94\x80 " :
"\xe2\x94\x9c\xe2\x94\x80\xe2\x94\x80 ") << children[i]->name << '\n';
            children[i]->print_children(os, prefix + (is_last ? " " : "\xe2\x94\x82  "));
        }
    }

    void print(std::ostream& os) const {
        os << name << '\n';
        print_children(os);
    }
};

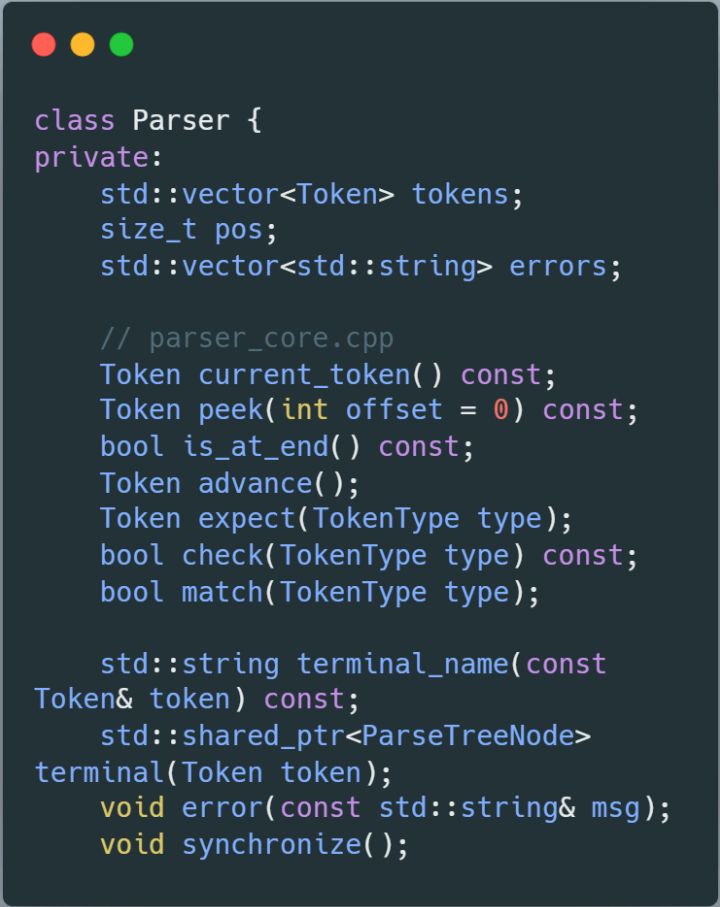
inline std::shared_ptr<ParseTreeNode> make_node(const std::string& name) {
    return std::make_shared<ParseTreeNode>(name);
}

```

2. Desain Kelas Parser

Kelas Parser menyimpan token list, posisi token saat ini, dan daftar error. Field tokens menyimpan token hasil lexer yang sudah difilter, pos menunjukkan token yang sedang dibaca, dan errors menyimpan semua syntax error. Parser juga memiliki helper seperti `current_token()`, `peek()`, `advance()`, `check()`, `match()`, `expect()`, `terminal()`, `error()`, dan `synchronize()`.

parser.hpp



```
class Parser {
private:
    std::vector<Token> tokens;
    size_t pos;
    std::vector<std::string> errors;

    // parser_core.cpp
    Token current_token() const;
    Token peek(int offset = 0) const;
    bool is_at_end() const;
    Token advance();
    Token expect(TokenType type);
    bool check(TokenType type) const;
    bool match(TokenType type);

    std::string terminal_name(const
Token& token) const;
    std::shared_ptr<ParseTreeNode>
terminal(Token token);
    void error(const std::string& msg);
    void synchronize();
};
```

Menunjukkan atribut utama parser, yaitu tokens, pos, dan errors, serta helper dasar untuk membaca token dan menangani error.

parser_core.cpp

```

Token Parser::current_token() const {
    if (!tokens.empty() && pos <
tokens.size()) return tokens[pos];
    if (!tokens.empty()) return
tokens.back();
    return Token(TokenType::TOKEN_EOF,
"", 0, 0);
}

Token Parser::peek(int offset) const {
    if (offset < 0) {
        const size_t distance =
static_cast<size_t>(-offset);
        if (distance > pos ||
tokens.empty()) {
            return
Token(TokenType::TOKEN_EOF, "", 0, 0);
        }
        return tokens[pos - distance];
    }

    size_t idx = pos +
static_cast<size_t>(offset);
    if (!tokens.empty() && idx <
tokens.size()) return tokens[idx];
    if (!tokens.empty()) return
tokens.back();
    return Token(TokenType::TOKEN_EOF,
"", 0, 0);
}

bool Parser::is_at_end() const {
    return tokens.empty() ||
pos >= tokens.size() ||
current_token().get_type()
== TokenType::TOKEN_EOF;
}

```

```

Token Parser::advance() {
    Token token = current_token();
    if (!is_at_end()) ++pos;
    return token;
}

bool Parser::check(TokenType type)
const {
    return current_token().get_type()
== type;
}

bool Parser::match(TokenType type) {
    if (!check(type)) return false;
    advance();
    return true;
}

Token Parser::expect(TokenType type) {
    if (check(type)) return advance();

    Token token = current_token();
    error("Syntax error at line " +
std::to_string(token.get_line()) +
        ", col " +
std::to_string(token.get_column()) +
        ": unexpected token " +
Token::get_type_name(token.get_type())
+
        ", expected " +
Token::get_type_name(type));

    return Token(type, "<missing>",
token.get_line(), token.get_column());
}

```

Menunjukkan implementasi helper inti parser seperti `current_token()`, `peek()`, `advance()`, `check()`, `match()`, dan `expect()`. Helper ini dipakai oleh semua fungsi grammar.

3. Integrasi Main Program

Main program menjadi penghubung antara lexer dan parser. Program menerima path file input, menjalankan lexer, memeriksa `TOKEN_ERROR`, lalu menghentikan parsing jika ada lexical error. Setelah itu, `TOKEN_COMMENT` dan `TOKEN_NEWLINE` difilter. Token hasil filter diberikan ke Parser untuk menghasilkan parse tree.

main.cpp

```
Lexer lexer(input_file);
vector<Token> tokens =
lexer.tokenize();

bool has_lexer_error = false;
for (const auto& token : tokens) {
    if (token.get_type() ==
TokenType::TOKEN_ERROR) {
        has_lexer_error = true;
        cerr << "Lexical error at
line " << token.get_line()
            << ", col " <<
token.get_column()
            << ": " <<
token.get_value() << endl;
    }
}

if (has_lexer_error) {
    cerr << "Parsing aborted due to
lexical errors." << endl;
    return 1;
}

vector<Token> filtered;
for (const auto& token : tokens) {
    if (token.get_type() !=
TokenType::TOKEN_COMMENT &&
        token.get_type() !=
TokenType::TOKEN_NEWLINE) {
        filtered.push_back(token);
    }
}

Parser parser(filtered);
auto tree = parser.parse();

tree->print(cout);
```

Menunjukkan alur integrasi lexer ke parser. Program menjalankan lexer, memeriksa TOKEN_ERROR, memfilter TOKEN_COMMENT dan TOKEN_NEWLINE, lalu membuat objek Parser dan memanggil parse().

main.cpp

```
string filename = input_file;
size_t last_slash =
filename.find_last_of("\\\\");
if (last_slash != string::npos) {
    filename =
filename.substr(last_slash + 1);
}
string output_path =
"test/milestone-2/" + filename +
"_OUTPUT.txt";

ofstream out(output_path);
if (out.is_open()) {
    tree->print(out);
    out.close();
} else {
    cerr << "Warning: Could not
open output file: " << output_path <<
endl;
}

const auto& errors =
parser.get_errors();
if (!errors.empty()) {
    cerr << endl << "Parser errors
(" << errors.size() << "):" << endl;
    for (const auto& err : errors)
    {
        cerr << " " << err <<
endl;
    }
    return 1;
}
```

Menunjukkan hasil parse tree dicetak ke terminal dan disimpan ke file output. Bagian ini juga menampilkan parser error jika ditemukan kesalahan sintaksis.

4. Implementasi Top-Level Grammar

Top-level parser menangani struktur utama program, yaitu `<program>`, `<program-header>`, `<declaration-part>`, `<block>`, `<compound-statement>`, dan `<statement-list>`. Fungsi `parse_program()` membentuk urutan utama program: program header, declaration part, compound statement, dan period. Parser juga mengecek apakah masih ada token setelah period.

parser_toplevel.cpp

```
std::shared_ptr<ParseTreeNode>
Parser::parse_program() {
    auto node = make_node("<program>");

    node-
>add_child(parse_program_header());
    node-
>add_child(parse_declaration_part());
    node-
>add_child(parse_compound_statement());
    node-
>add_child(terminal(expect(TokenType::T
OKEN_PERIOD)));

    if (!is_at_end()) {
        Token token = current_token();
        error("Syntax error at line " +
std::to_string(token.get_line()) +
", col " +
std::to_string(token.get_column()) +
": unexpected token " +
Token::get_type_name(token.get_type())
+
" after end of program");
    }

    return node;
}
```

Menunjukkan implementasi <program>. Fungsi parse_program() membentuk urutan utama program: program header, declaration part, compound statement, dan period.

Fungsi parse_declaration_part() memastikan urutan deklarasi sesuai spesifikasi, yaitu const, type, var, lalu subprogram. Jika deklarasi muncul di luar urutan tersebut, parser mencatat syntax error.

parser_toplevel.cpp

```
std::shared_ptr<ParseTreeNode>
Parser::parse_declaration_part() {
    auto node = make_node("
<declaration-part>");

    auto declaration_rank = []
(TokenTypes type) -> int {
        if (type ==
TokenType::TOKEN_CONSTSY) return 0;
        if (type ==
TokenType::TOKEN_TYPESY) return 1;
        if (type ==
TokenType::TOKEN_VARSY) return 2;
        if (type ==
TokenType::TOKEN_PROCEDESY || type ==
TokenType::TOKEN_FUNCTIONSY) return 3;
        return -1;
    };

    auto append_declaration = [&](const
std::shared_ptr<ParseTreeNode>& child,
const
std::string& rule,

size_t before) {
        node->add_child(child);
        if (pos == before &&
!is_at_end()) {
            Token token =
current_token();
            error("Syntax error at line
" + std::to_string(token.get_line()) +
", col " +
std::to_string(token.get_column()) +
": " + rule + " did
not consume token " +
Token::get_type_name(token.get_type())
+
"; advancing to
continue parsing");
            advance();
        }
    };
};
```

```

int current_rank = 0;
while (!is_at_end()) {
    TokenType type =
current_token().get_type();
    const int rank =
declaration_rank(type);
    if (rank == -1) break;

    if (rank < current_rank) {
        Token token =
current_token();
        error("Syntax error at line
" + std::to_string(token.get_line()) +
            ", col " +
std::to_string(token.get_column()) +
            ": declaration '" +
Token::get_type_name(token.get_type())
+
            "' appears out of
order (expected order: const, type,
var, subprogram)");
    } else {
        current_rank = rank;
    }

    const size_t before = pos;
    if (type ==
TokenType::TOKEN_CONSTSY) {
        append_declaration(parse_const_declarati
on(), "<const-declaration>", before);
    } else if (type ==
TokenType::TOKEN_TYPESY) {
        append_declaration(parse_type_declarati
on(), "<type-declaration>", before);
    } else if (type ==
TokenType::TOKEN_VARSY) {
        append_declaration(parse_var_declarati
on(), "<var-declaration>", before);
    } else {
        append_declaration(parse_subprogram_de
claration(), "<subprogram-
declaration>", before);
    }
}

return node;
}

```

Menunjukkan implementasi <declaration-part>. Parser memastikan deklarasi muncul sesuai urutan wajib: const, type, var, lalu subprogram.

5. Implementasi Declaration Parser

Declaration parser menangani deklarasi const, type, var, identifier list, array type, range, enumerated, record type, field list, dan field part. Fungsi parse_type() melakukan disambiguasi berdasarkan token awal. Jika token adalah arraysy, parser memanggil parse_array_type(); jika lparent, parser memanggil parse_enumerated(); jika recordsy, parser memanggil parse_record_type(); dan jika ditemukan pola constant period period constant, parser memanggil parse_range().

parser_declarations.cpp

```
std::shared_ptr<ParseTreeNode>
Parser::parse_type() {
    auto node = make_node("<type>");
    const TokenType type =
current_token().get_type();

    if (type ==
TokenType::TOKEN_ARRAYSY) {
        node->
>add_child(parse_array_type());
        return node;
    }

    if (type ==
TokenType::TOKEN_LPARENT) {
        node->
>add_child(parse_enumerated());
        return node;
    }

    if (type ==
TokenType::TOKEN_RECORDSY) {
        node->
>add_child(parse_record_type());
        return node;
    }

    if (starts_constant(type)) {
        const int next_after_constant =
is_sign(type) ? 2 : 1;
        if
(peek(next_after_constant).get_type()
== TokenType::TOKEN_PERIOD &&
peek(next_after_constant +
1).get_type() ==
TokenType::TOKEN_PERIOD) {
            node->
>add_child(parse_range());
            return node;
        }
    }
}
```

```

        if (type == TokenType::TOKEN_IDENT)
        {
            node-
>add_child(terminal(advance()));
            return node;
        }

        error(syntax_error_at(current_token(),
"\"type\""));
            return node;
        }

```

Menunjukkan implementasi `parse_type()`. Fungsi ini memilih jenis `type` berdasarkan token awal, seperti array, enumerated, record, range, atau identifier.

parser_declarations.cpp

```

std::shared_ptr<ParseTreeNode>
Parser::parse_array_type() {
    auto node = make_node("<array-
type>");

    node-
>add_child(terminal(expect(TokenType::T
OKEN_ARRAYSY)));
    node-
>add_child(terminal(expect(TokenType::T
OKEN_LBRACK)));

    if (check(TokenType::TOKEN_IDENT)
&& peek(1).get_type() !=
TokenType::TOKEN_PERIOD) {
        node-
>add_child(terminal(advance()));
    } else {
        node->add_child(parse_range());
    }

    node-
>add_child(terminal(expect(TokenType::T
OKEN_RBRACK)));
    node-
>add_child(terminal(expect(TokenType::T
OKEN_OFSY)));
    node->add_child(parse_type());

    return node;
}

```

```

std::shared_ptr<ParseTreeNode>
Parser::parse_range() {
    auto node = make_node("<range>");

    node->add_child(parse_constant());
    node-
>add_child(terminal(expect(TokenType::T
OKEN_PERIOD)));
    node-
>add_child(terminal(expect(TokenType::T
OKEN_PERIOD)));
    node->add_child(parse_constant());

    return node;
}

std::shared_ptr<ParseTreeNode>
Parser::parse_enumerated() {
    auto node = make_node("
<enumerated>");

    node-
>add_child(terminal(expect(TokenType::T
OKEN_LPARENT)));
    node-
>add_child(terminal(expect(TokenType::T
OKEN_IDENT)));

    while
    (check(TokenType::TOKEN_COMMA)) {
        node-
>add_child(terminal(advance()));
        node-
>add_child(terminal(expect(TokenType::T
OKEN_IDENT)));
    }

    node-
>add_child(terminal(expect(TokenType::T
OKEN_RPARENT)));

    return node;
}

```

Menunjukkan implementasi array type, range, dan enumerated type. Bagian ini mendukung bentuk seperti array[Range] of integer, 1..10, dan (Jan, Feb, Mar).

6. Implementasi Statement dan Subprogram Parser

Statement parser menangani subprogram declaration, procedure, function, parameter formal, statement, variable chain, index list, assignment, dan if. Fungsi parse_statement() memilih jenis statement berdasarkan token awal. Statement kosong tetap valid. Identifier yang diikuti lpARENT diperlakukan sebagai procedure/function call, sedangkan identifier lain diproses sebagai variable-like statement atau assignment berdasarkan lookahead setelah variable chain.

```

std::shared_ptr<ParseTreeNode>
Parser::parse_subprogram_declaration()
{
    auto node = make_node("<subprogram-
declaration>");

    if
    (check(TokenType::TOKEN_PROCEDURES)) {
        node-
>add_child(parse_procedure_declaration(
));
    } else if
    (check(TokenType::TOKEN_FUNCTIONS)) {
        node-
>add_child(parse_function_declaration()
);
    } else {
        Token token = current_token();
        error("Syntax error at line " +
std::to_string(token.get_line()) +
        ", col " +
std::to_string(token.get_column()) +
        ": unexpected token " +
Token::get_type_name(token.get_type())
+
        ", expected proceduresy
or functionsy");
        node->add_child(make_node("<
error>"));
        synchronize();
    }

    return node;
}

std::shared_ptr<ParseTreeNode>
Parser::parse_procedure_declaration() {
    auto node = make_node("<procedure-
declaration>");

    node-
>add_child(terminal(expect(TokenType::T
OKEN_PROCEDURES)));
    node-
>add_child(terminal(expect(TokenType::T
OKEN_IDENT)));

    if
    (check(TokenType::TOKEN_LPARENT)) {
        node-
>add_child(parse_formal_parameter_list(
));
    }

    node-
>add_child(terminal(expect(TokenType::T
OKEN_SEMICOLON)));
    node->add_child(parse_block());
    node-
>add_child(terminal(expect(TokenType::T
OKEN_SEMICOLON)));

    return node;
}

```

```

std::shared_ptr<ParseTreeNode>
Parser::parse_function_declaration() {
    auto node = make_node("<function-
declaration>");

    node-
>add_child(terminal(expect(TokenType::T
OKEN_FUNCTIONSY)));
    node-
>add_child(terminal(expect(TokenType::T
OKEN_IDENT)));

    if
    (check(TokenType::TOKEN_LPARENT)) {
        node-
>add_child(parse_formal_parameter_list(
));
    }

    node-
>add_child(terminal(expect(TokenType::T
OKEN_COLON)));
    node-
>add_child(terminal(expect(TokenType::T
OKEN_IDENT)));
    node-
>add_child(terminal(expect(TokenType::T
OKEN_SEMICOLON)));
    node->add_child(parse_block());
    node-
>add_child(terminal(expect(TokenType::T
OKEN_SEMICOLON)));

    return node;
}

```

Menunjukkan implementasi subprogram declaration. Parser membedakan procedure dan function declaration, lalu memproses parameter formal dan block.

parser_statements.cpp


```

std::shared_ptr<ParseTreeNode>
Parser::parse_statement() {
    auto node = make_node("
<statement>");

    if (check(TokenType::TOKEN_IFSY)) {
        node-
>add_child(parse_if_statement());
    } else if
(check(TokenType::TOKEN_CASESY)) {
        add_child_or_placeholder(node,
parse_case_statement(), "<case-
statement>");
    } else if
(check(TokenType::TOKEN_WHILESY)) {
        add_child_or_placeholder(node,
parse_while_statement(), "<while-
statement>");
    } else if
(check(TokenType::TOKEN_REPEATSY)) {
        add_child_or_placeholder(node,
parse_repeat_statement(), "<repeat-
statement>");
    } else if
(check(TokenType::TOKEN_FORSY)) {
        add_child_or_placeholder(node,
parse_for_statement(), "<for-
statement>");
    } else if
(check(TokenType::TOKEN_BEGINSY)) {
        node-
>add_child(parse_compound_statement());
    } else if
(check(TokenType::TOKEN_IDENT)) {
        if (peek(1).get_type() ==
TokenType::TOKEN_LPARENT) {

            add_child_or_placeholder(node,
parse_procedure_function_call(),
"
<procedure/function-call>");
        } else {
            size_t offset = 1;

            while (true) {
                TokenType type =
peek(static_cast<int>
(offset)).get_type();

                if (type ==
TokenType::TOKEN_LBRACK) {
                    int depth = 0;

                    do {
                        TokenType
bracket_type = peek(static_cast<int>
(offset)).get_type();

```

```

        if
        (bracket_type ==
TokenType::TOKEN_LBRACK) {
            ++depth;
        } else if
        (bracket_type ==
TokenType::TOKEN_RBRACK) {
            --depth;
        }

        ++offset;

        if
        (bracket_type == TokenType::TOKEN_EOF
||
            (depth > 0
&&
is_statement_boundary(bracket_type))) {
            break;
        }
    } while (depth >
0);
    } else if (type ==
TokenType::TOKEN_PERIOD &&
peek(static_cast<int>(offset +
1)).get_type() ==
TokenType::TOKEN_IDENT) {
        offset += 2;
    } else {
        break;
    }
}

    if (peek(static_cast<int>
(offset)).get_type() ==
TokenType::TOKEN_BECOMES) {
        node-
>add_child(parse_assignment_statement()
);
    } else {
        node-
>add_child(parse_variable());
    }
}
} else if
(!is_statement_boundary(current_token()
.get_type())) {
    Token token = current_token();
    error("Syntax error at line " +
std::to_string(token.get_line()) +
", col " +
std::to_string(token.get_column()) +
": unexpected token " +
Token::get_type_name(token.get_type())
+
", expected statement");
    node->add_child(make_node("
<error>"));
}

```

Menunjukkan implementasi statement dispatch. Parser memilih jenis statement berdasarkan token awal, menangani statement kosong, procedure/function call, variable-like statement, dan assignment.

7. Implementasi Expression dan Control Flow Parser

Expression parser menangani case, while, repeat, for, procedure/function call, parameter list, expression, simple expression, term, factor, dan operator wrapper. Precedence operator dijaga dengan memecah expression menjadi beberapa level: expression untuk relational operator, simple_expression untuk additive operator, term untuk multiplicative operator, dan factor untuk operand dasar.

parser_expressions.cpp
<pre>std::shared_ptr<ParseTreeNode> Parser::parse_case_statement() { auto node = make_node("<case- statement>"); node- >add_child(terminal(expect(TokenType::T OKEN_CASESY))); node- >add_child(parse_expression()); node- >add_child(terminal(expect(TokenType::T OKEN_OFSY))); node- >add_child(parse_case_block()); node- >add_child(terminal(expect(TokenType::T OKEN_ENDSY))); return node; } std::shared_ptr<ParseTreeNode> Parser::parse_case_block() { auto node = make_node("<case- block>"); if (!is_constant_start(current_token().get _type())) { error(expected_message(current_token() , "constant")); node->add_child(make_node("<error>")); return node; } node->add_child(parse_constant()); while (check(token_type::TOKEN_COMMA)) { node- >add_child(terminal(advance())); if (!is_constant_start(current_token().get _type())) { error(expected_message(current_token() , "constant")); node->add_child(make_node("<error>")); break; } } }</pre>

```

        node->add_child(parse_constant());
    }

    node->add_child(terminal(expect(TokenType::TOKEN_COLON)));
    node->add_child(parse_statement());

    while
    {check(TokenType::TOKEN_SEMICOLON)} {
        node->add_child(terminal(advance()));

        if
        {check(TokenType::TOKEN_ENDSY) ||
         is_at_end()} {
            break;
        }

        if
        {is_constant_start(current_token().get_type())} {
            node->add_child(parse_case_block());
            break;
        }

        error(expected_message(current_token()),
              "case label or endsy");
        node->add_child(make_node("<error>"));
        if
        {!check(TokenType::TOKEN_SEMICOLON) &&
         !is_at_end()} {
            advance();
        }
    }

    return node;
}

std::shared_ptr<ParseTreeNode>
Parser::parse_while_statement() {
    auto node = make_node("<while-statement>");

    node->add_child(terminal(expect(TokenType::TOKEN_WHILESY)));
    node->add_child(parse_expression());
    node->add_child(terminal(expect(TokenType::TOKEN_DOSY)));
    node->add_child(parse_statement());

    return node;
}

std::shared_ptr<ParseTreeNode>
Parser::parse_repeat_statement() {
    auto node = make_node("<repeat-statement>");

    node->add_child(terminal(expect(TokenType::TOKEN_REPEATSY)));
    node->add_child(parse_statement_list());
    node->add_child(terminal(expect(TokenType::TOKEN_UNILSY)));
    node->add_child(parse_expression());

    return node;
}

```

```

std::shared_ptr<ParseTreeNode>
Parser::parse_for_statement() {
    auto node = make_node("<for-
statement>");

    node-
>add_child(terminal(expect(TokenType::T
OKEN_FORSY)));
    node-
>add_child(terminal(expect(TokenType::T
OKEN_IDENT)));
    node-
>add_child(terminal(expect(TokenType::T
OKEN_BECOMES)));
    node-
>add_child(parse_expression());

    if (check(TokenType::TOKEN_TOSY) ||
check(TokenType::TOKEN_DOWNTOSY)) {
        node-
>add_child(terminal(advance()));
    } else {
        error(expected_message(current_token()
, "tosy or downtosy"));
        node->add_child(make_node("
<error>"));
    }

    node-
>add_child(parse_expression());
    node-
>add_child(terminal(expect(TokenType::T
OKEN_DOSY)));
    node->add_child(parse_statement());

    return node;
}

```

Menunjukkan implementasi control flow seperti case, while, repeat, dan for. Setiap struktur kontrol dipetakan ke fungsi recursive descent masing-masing.

parser_expressions.cpp

```

std::shared_ptr<ParseTreeNode>
Parser::parse_procedure_function_call()
{
    auto node = make_node("
<procedure/function-call>");

    node-
>add_child(terminal(expect(TokenType::T
OKEN_IDENT)));
    node-
>add_child(terminal(expect(TokenType::T
OKEN_LPARENT)));

    if
(check(TokenType::TOKEN_RPARENT)) {

        error(expected_message(current_token()
, "<parameter-list>"));
        node->add_child(make_node("
<parameter-list>"));
    } else {
        node-
>add_child(parse_parameter_list());
    }

    node-
>add_child(terminal(expect(TokenType::T
OKEN_RPARENT)));

    return node;
}

std::shared_ptr<ParseTreeNode>
Parser::parse_parameter_list() {
    auto node = make_node("<parameter-
list>");

    if
(!is_expression_start(current_token().g
et_type())) {

        error(expected_message(current_token()
, "expression"));
        node->add_child(make_node("
<error>"));
        if
(!check(TokenType::TOKEN_RPARENT) &&
!is_at_end()) {
            node-
>add_child(terminal(advance()));
        }
        return node;
    }
}

```

```

    node-
    >add_child(parse_expression());

    while
    (check(TokenType::TOKEN_COMMA)) {
        node-
        >add_child(terminal(advance()));

        if
        (!is_expression_start(current_token().get_type())) {

            error(expected_message(current_token()), "expression");
            node->add_child(make_node("<error>"));
            if
            (!check(TokenType::TOKEN_RPARENT) &&
             !is_at_end()) {
                node-
                >add_child(terminal(advance()));
            }
            break;
        }

        node-
        >add_child(parse_expression());
    }

    return node;
}

```

Menunjukkan implementasi <procedure/function-call> dan <parameter-list>. Parser memastikan call menggunakan tanda kurung dan parameter list sesuai grammar.

parser_expressions.cpp

```

std::shared_ptr<ParseTreeNode>
Parser::parse_expression() {
    auto node = make_node("<expression>");

    node->add_child(parse_simple_expression());

    if
(is_relational_operator(current_token()
.get_type())) {
        node->add_child(parse_relational_operator());
    }

    node->add_child(parse_simple_expression());

    return node;
}

std::shared_ptr<ParseTreeNode>
Parser::parse_simple_expression() {
    auto node = make_node("<simple-expression>");

    if (check(TokenType::TOKEN_PLUS) ||
check(TokenType::TOKEN_MINUS)) {
        node->add_child(terminal(advance()));

        node->add_child(parse_term());

        while
(is_additive_operator(current_token().get_type())) {
            node->add_child(parse_additive_operator());
            node->add_child(parse_term());
        }

        return node;
    }

    std::shared_ptr<ParseTreeNode>
    Parser::parse_term() {
        auto node = make_node("<term>");

        node->add_child(parse_factor());

        while
(is_multiplicative_operator(current_token().get_type())) {
            node->add_child(parse_multiplicative_operator());
            node->add_child(parse_factor());
        }

        return node;
    }
}

```



```

std::shared_ptr<ParseTreeNode>
Parser::parse_factor() {
    auto node = make_node("<factor>");
    Token token = current_token();

    if (token.get_type() ==
TokenType::TOKEN_IDENT) {
        if (peek(1).get_type() ==
TokenType::TOKEN_LPARENT) {
            node-
>add_child(parse_procedure_function_cal
l());
        } else {
            node-
>add_child(parse_variable());
        }
    } else if (token.get_type() ==
TokenType::TOKEN_INTCON ||
token.get_type() ==
TokenType::TOKEN_REALCON ||
token.get_type() ==
TokenType::TOKEN_CHARCON ||
token.get_type() ==
TokenType::TOKEN_STRING) {
        node-
>add_child(terminal(advance()));
    } else if (token.get_type() ==
TokenType::TOKEN_LPARENT) {
        node-
>add_child(terminal(advance()));
        node-
>add_child(parse_expression());
        node-
>add_child(terminal(expect(TokenType::T
OKEN_RPARENT)));
    } else if (token.get_type() ==
TokenType::TOKEN_NOTSY) {
        node-
>add_child(terminal(advance()));
        node-
>add_child(parse_factor());
    } else {
        error(expected_message(token,
"factor"));
        node->add_child(make_node("
<error>"));
        if
(!is_factor_recovery_token(token.get_ty
pe()) && !is_at_end()) {
            node-
>add_child(terminal(advance()));
        }
    }

    return node;
}

```

Menunjukkan implementasi expression precedence. Parser memecah expression menjadi simple-expression, term, dan factor agar prioritas operator diproses dengan benar.

parser_expressions.cpp

```

std::shared_ptr<ParseTreeNode>
Parser::parse_relational_operator() {
    auto node = make_node("<relational-
operator>");

    if
(is_relational_operator(current_token()
.get_type())) {
        node-
>add_child(terminal(advance()));
    } else {

        error(expected_message(current_token()
, "relational operator"));
        node->add_child(make_node("
<error>"));
    }

    return node;
}

std::shared_ptr<ParseTreeNode>
Parser::parse_additive_operator() {
    auto node = make_node("<additive-
operator>");

    if
(is_additive_operator(current_token().g
et_type())) {
        node-
>add_child(terminal(advance()));
    } else {

        error(expected_message(current_token()
, "additive operator"));
        node->add_child(make_node("
<error>"));
    }

    return node;
}

std::shared_ptr<ParseTreeNode>
Parser::parse_multiplicative_operator()
{
    auto node = make_node("
<multiplicative-operator>");

    if
(is_multiplicative_operator(current_tok
en().get_type())) {
        node-
>add_child(terminal(advance()));
    } else {

        error(expected_message(current_token()
, "multiplicative operator"));
        node->add_child(make_node("
<error>"));
    }

    return node;
}

```

Menunjukkan implementasi wrapper node untuk relational, additive, dan multiplicative operator. Wrapper node ini dipertahankan agar parse tree sesuai grammar.

8. Strategi Error Handling

Error handling parser dilakukan dengan mengumpulkan pesan error dalam vector errors. Fungsi expect() mencatat error ketika token aktual tidak sesuai dengan token yang diharapkan. Parser juga memiliki synchronize() untuk berpindah ke token aman seperti semicolon, endsy, elsesy, untilsy, ofsy, atau period agar parser tidak terjebak infinite loop. Pada parse tree, node error atau token missing dapat digunakan agar proses pembentukan tree tetap berlanjut.



Menunjukkan fungsi expect() yang mencatat syntax error ketika token aktual tidak sesuai token yang diharapkan. Error mencakup baris, kolom, token aktual, dan expected token.

```
void Parser::error(const std::string&
msg) {
    errors.push_back(msg);
}

const std::vector<std::string>&
Parser::get_errors() const {
    return errors;
}

void Parser::synchronize() {
    while (!is_at_end()) {
        if
        (check(TokenType::TOKEN_SEMICOLON) ||
         check(TokenType::TOKEN_ENDSY) ||
         check(TokenType::TOKEN_ELSESY) ||
         check(TokenType::TOKEN_UNTILSY) ||
         check(TokenType::TOKEN_OFSY) ||
         check(TokenType::TOKEN_PERIOD)) {
            return;
        }
        advance();
    }
}
```

Menunjukkan penyimpanan error dan recovery sederhana melalui synchronize(). Fungsi ini membantu parser lanjut ke token aman agar tidak infinite loop.

III. Pengujian

Test Case / Input / Output

Test Case 1 : tes struktur program valid yang paling dasar: header program, blok begin-end yang kosong, dan titik penutup

```
test > milestone-2 > ≡ tc1.txt
```

```
1  program Minimal;  
2  begin  
3  end.  
4
```

```
test > milestone-2 > ≡ tc1.txt_OUTPUT.txt
```

```
1  <program>  
2  |— <program-header>  
3  |   |— programsy  
4  |   |— ident(Minimal)  
5  |   |— semicolon  
6  |— <declaration-part>  
7  |— <compound-statement>  
8  |   |— beginsy  
9  |   |— <statement-list>  
10  |   |   |— <statement>  
11  |   |   |— endsy  
12  |— period  
13
```

Test Case 2: tes kemampuan parser dalam menangani semua blok deklarasi (const, type, var) dengan benar sesuai urutan yang diwajibkan, termasuk tipe subrange dan enumerasi (enumerated)

```
test > milestone-2 > ≡ tc2.txt
1  program Declarations;
2  √ const
3    PI == 3.14;
4    LIMIT == 100;
5  √ type
6    TIndex == 1..10;
7    TColor == (Red, Green, Blue);
8  √ var
9    i, j: integer;
10   flag: boolean;
11  √ begin
12    i := 1;
13   end.
14
```

```

test > milestone-2 > tc2.txt_OUTPUT.txt
1  <program>
2  |<program-header>
3  |<programs>
4  |<ident(Declarations)>
5  |<semicolon>
6  |<declaration-part>
7  |<const-declaration>
8  |<constsy>
9  |<ident(PI)>
10 |<eq1>
11 |<constant>
12 |<realcon(3.14)>
13 |<semicolon>
14 |<ident(LIMIT)>
15 |<eq1>
16 |<constant>
17 |<intcon(100)>
18 |<semicolon>
19 |<type-declaration>
20 |<typesy>
21 |<ident(TIndex)>
22 |<eq1>
23 |<type>
24 |<range>
25 |<constant>
26 |<intcon(1)>
27 |<period>
28 |<period>
29 |<constant>
30 |<intcon(10)>
31 |<semicolon>
32 |<ident(TColor)>
33 |<eq1>
34 |<type>
35 |<enumerated>
36 |<lparent>
37 |<ident(Red)>
38 |<comma>
39 |<ident(Green)>
40 |<comma>
41 |<ident(Blue)>
42 |<rparent>
43 |<semicolon>
44 |<var-declaration>
45 |<varsy>
46 |<identifier-list>
47 |<ident(i)>
48 |<comma>
49 |<ident(j)>
50 |<colon>
51 |<type>
52 |<ident(integer)>
53 |<semicolon>
54 |<identifier-list>
55 |<ident(flag)>
56 |<colon>
57 |<type>
58 |<ident(boolean)>
59 |<semicolon>
60 |<compound-statement>
61 |<beginsy>
62 |<statement-list>
63 |<statement>
64 |<assignment-statement>
65 |<variable>
66 |<ident(i)>
67 |<becomes>
68 |<expression>
69 |<simple-expression>
70 |<term>
71 |<factor>
72 |<intcon(1)>
73 |<semicolon>
74 |<statement>
75 |<endsy>
76 |<period>
77

```

Test Case 3: tes berbagai deklarasi prosedur dan fungsi beserta parameter formal dan pelingkupan (scoping) blok yang benar

```
test > milestone-2 > ≡ tc3.txt
1  program Subprograms;
2  procedure PrintVal(x: integer);
3  var
4  |   localCount: integer;
5  begin
6  |   localCount := x;
7  |   writeln(localCount);
8  end;
9  procedure SwapLike(a, b: integer);
10 var
11 |   temp: integer;
12 begin
13 |   temp := a;
14 |   a := b;
15 |   b := temp;
16 end;
17 function Add(a, b: integer): integer;
18 var
19 |   total: integer;
20 begin
21 |   total := a + b;
22 |   Add := total;
23 end;
24 function Twice(n: integer): integer;
25 begin
26 |   Twice := Add(n, n);
27 end;
28 begin
29 |   PrintVal(Twice(5));
30 |   SwapLike(1, 2);
31 end.
32
```


test > milestone-2 > tc3.txt_OUTPUT.txt

```
1  <program>
2  |  <program-header>
3  |  |  programsy
4  |  |  ident(Subprograms)
5  |  |  semicolon
6  |  <declaration-part>
7  |  |  <subprogram-declaration>
8  |  |  |  <procedure-declaration>
9  |  |  |  |  proceduresy
10 |  |  |  |  ident(PrintVal)
11 |  |  |  |  <formal-parameter-list>
12 |  |  |  |  |  lparent
13 |  |  |  |  |  <parameter-group>
14 |  |  |  |  |  |  <identifier-list>
15 |  |  |  |  |  |  |  ident(x)
16 |  |  |  |  |  |  colon
17 |  |  |  |  |  |  ident(integer)
18 |  |  |  |  |  rparent
19 |  |  |  |  semicolon
20 |  |  |  <block>
21 |  |  |  |  <declaration-part>
22 |  |  |  |  |  <var-declaration>
23 |  |  |  |  |  |  varsy
24 |  |  |  |  |  |  <identifier-list>
25 |  |  |  |  |  |  |  ident(localCount)
26 |  |  |  |  |  |  colon
27 |  |  |  |  |  |  <type>
28 |  |  |  |  |  |  |  ident(integer)
29 |  |  |  |  |  |  semicolon
30 |  |  |  |  <compound-statement>
31 |  |  |  |  |  beginsy
32 |  |  |  |  |  <statement-list>
33 |  |  |  |  |  |  <statement>
34 |  |  |  |  |  |  |  <assignment-statement>
35 |  |  |  |  |  |  |  |  <variable>
36 |  |  |  |  |  |  |  |  |  ident(localCount)
37 |  |  |  |  |  |  |  |  becomes
38 |  |  |  |  |  |  |  |  <expression>
39 |  |  |  |  |  |  |  |  |  <simple-expression>
40 |  |  |  |  |  |  |  |  |  <term>
41 |  |  |  |  |  |  |  |  |  |  <factor>
42 |  |  |  |  |  |  |  |  |  |  |  <variable>
43 |  |  |  |  |  |  |  |  |  |  |  |  ident(x)
44 |  |  |  |  |  |  |  |  |  |  semicolon
45 |  |  |  |  |  |  |  |  <statement>
46 |  |  |  |  |  |  |  |  |  <procedure/function-call>
47 |  |  |  |  |  |  |  |  |  |  ident(writeln)
48 |  |  |  |  |  |  |  |  |  |  lparent
49 |  |  |  |  |  |  |  |  |  |  <parameter-list>
50 |  |  |  |  |  |  |  |  |  |  |  <expression>
51 |  |  |  |  |  |  |  |  |  |  |  |  <simple-expression>
52 |  |  |  |  |  |  |  |  |  |  |  |  <term>
53 |  |  |  |  |  |  |  |  |  |  |  |  |  <factor>
54 |  |  |  |  |  |  |  |  |  |  |  |  |  |  <variable>
55 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  ident(localCount)
56 |  |  |  |  |  |  |  |  |  |  rparent
57 |  |  |  |  |  |  |  |  |  |  semicolon
58 |  |  |  |  |  |  |  |  |  <statement>
59 |  |  |  |  |  |  |  |  |  endsy
60 |  |  |  |  |  |  |  |  semicolon
61 |  |  |  <subprogram-declaration>
```

```

61 | └─ <subprogram-declaration>
62 |   └─ <procedure-declaration>
63 |     └─ proceduresy
64 |       └─ ident(SwapLike)
65 |         └─ <formal-parameter-list>
66 |           └─ lparent
67 |             └─ <parameter-group>
68 |               └─ <identifier-list>
69 |                 └─ ident(a)
70 |                   └─ comma
71 |                     └─ ident(b)
72 |                       └─ colon
73 |                         └─ ident(integer)
74 |                           └─ rparent
75 |                             └─ semicolon
76 |           └─ <block>
77 |             └─ <declaration-part>
78 |               └─ <var-declaration>
79 |                 └─ varsy
80 |                   └─ <identifier-list>
81 |                     └─ ident(temp)
82 |                       └─ colon
83 |                         └─ <type>
84 |                           └─ ident(integer)
85 |                             └─ semicolon
86 |             └─ <compound-statement>
87 |               └─ beginsy
88 |                 └─ <statement-list>
89 |                   └─ <statement>
90 |                     └─ <assignment-statement>
91 |                       └─ <variable>
92 |                         └─ ident(temp)
93 |                       └─ becomes
94 |                         └─ <expression>
95 |                           └─ <simple-expression>
96 |                             └─ <term>
97 |                               └─ <factor>
98 |                                 └─ <variable>
99 |                                   └─ ident(a)
100 |                     └─ semicolon
101 |                   └─ <statement>
102 |                     └─ <assignment-statement>
103 |                       └─ <variable>
104 |                         └─ ident(a)
105 |                       └─ becomes
106 |                         └─ <expression>
107 |                           └─ <simple-expression>
108 |                             └─ <term>
109 |                               └─ <factor>
110 |                                 └─ <variable>
111 |                                   └─ ident(b)
112 |                     └─ semicolon
113 |                   └─ <statement>
114 |                     └─ <assignment-statement>
115 |                       └─ <variable>
116 |                         └─ ident(b)
117 |                       └─ becomes
118 |                         └─ <expression>
119 |                           └─ <simple-expression>
120 |                             └─ <term>
121 |                               └─ <factor>
122 |                                 └─ <variable>
123 |                                   └─ ident(temp)
124 |                     └─ semicolon
125 |                   └─ <statement>
126 |                     └─ endsy
127 |                 └─ semicolon
128 |             └─ <subprogram-declaration>

```

```

128 | └─ <subprogram-declaration>
129 |   └─ <function-declaration>
130 |     └─ functiony
131 |       └─ ident(Add)
132 |         └─ <formal-parameter-list>
133 |           └─ lparent
134 |             └─ <parameter-group>
135 |               └─ <identifier-list>
136 |                 └─ ident(a)
137 |                   └─ comma
138 |                     └─ ident(b)
139 |                       └─ colon
140 |                         └─ ident(integer)
141 |                           └─ rparent
142 |                             └─ colon
143 |                               └─ ident(integer)
144 |                                 └─ semicolon
145 |                                   └─ <block>
146 |                                     └─ <declaration-part>
147 |                                       └─ <var-declaration>
148 |                                         └─ varsy
149 |                                           └─ <identifier-list>
150 |                                             └─ ident(total)
151 |                                               └─ colon
152 |                                                 └─ <type>
153 |                                                   └─ ident(integer)
154 |                                                     └─ semicolon
155 |                                   └─ <compound-statement>
156 |                                     └─ beginsy
157 |                                       └─ <statement-list>
158 |                                         └─ <statement>
159 |                                           └─ <assignment-statement>
160 |                                             └─ <variable>
161 |                                               └─ ident(total)
162 |                                                 └─ becomes
163 |                                                   └─ <expression>
164 |                                                     └─ <simple-expression>
165 |                                                       └─ <term>
166 |                                                         └─ <factor>
167 |                                                           └─ <variable>
168 |                                                             └─ ident(a)
169 |                                                           └─ <additive-operator>
170 |                                                             └─ plus
171 |                                                           └─ <term>
172 |                                                             └─ <factor>
173 |                                                               └─ <variable>
174 |                                                                 └─ ident(b)
175 |                                           └─ semicolon
176 |                                         └─ <statement>
177 |                                           └─ <assignment-statement>
178 |                                             └─ <variable>
179 |                                               └─ ident(Add)
180 |                                                 └─ becomes
181 |                                                   └─ <expression>
182 |                                                     └─ <simple-expression>
183 |                                                       └─ <term>
184 |                                                         └─ <factor>
185 |                                                           └─ <variable>
186 |                                                             └─ ident(total)
187 |                                           └─ semicolon
188 |                                         └─ <statement>
189 |                                           └─ endsy
190 |                                     └─ semicolon
191 |                                   └─ <subprogram-declaration>

```

```

191 |   └─ <subprogram-declaration>
192 |       └─ <function-declaration>
193 |           └─ functionsy
194 |           └─ ident(Twice)
195 |           └─ <formal-parameter-list>
196 |               └─ lparent
197 |                   └─ <parameter-group>
198 |                       └─ <identifier-list>
199 |                           └─ ident(n)
200 |                       └─ colon
201 |                       └─ ident(integer)
202 |                   └─ rparent
203 |           └─ colon
204 |           └─ ident(integer)
205 |           └─ semicolon
206 |           └─ <block>
207 |               └─ <declaration-part>
208 |               └─ <compound-statement>
209 |                   └─ beginsy
210 |                   └─ <statement-list>
211 |                       └─ <statement>
212 |                           └─ <assignment-statement>
213 |                               └─ <variable>
214 |                                   └─ ident(Twice)
215 |                               └─ becomes
216 |                               └─ <expression>
217 |                                   └─ <simple-expression>
218 |                                       └─ <term>
219 |                                           └─ <factor>
220 |                                               └─ <procedure/function-call>
221 |                                                   └─ ident(Add)
222 |                                                   └─ lparent
223 |                                                       └─ <parameter-list>
224 |                                                           └─ <expression>
225 |                                                               └─ <simple-expression>
226 |                                                                   └─ <term>
227 |                                                                       └─ <factor>
228 |                                                                           └─ <variable>
229 |                                                                               └─ ident(n)
230 |                                                       └─ comma
231 |                                                       └─ <expression>
232 |                                                           └─ <simple-expression>
233 |                                                               └─ <term>
234 |                                                                   └─ <factor>
235 |                                                                       └─ <variable>
236 |                                                                           └─ ident(n)
237 |                                               └─ rparent
238 |                               └─ semicolon
239 |                               └─ <statement>
240 |                           └─ endsy
241 |                   └─ semicolon
242 |           └─ <compound-statement>

```

```

242 |— <compound-statement>
243 |   |— beginsy
244 |   |— <statement-list>
245 |   |   |— <statement>
246 |   |       |— <procedure/function-call>
247 |   |           |— ident(PrintVal)
248 |   |               |— lparent
249 |   |                   |— <parameter-list>
250 |   |                       |— <expression>
251 |   |                           |— <simple-expression>
252 |   |                               |— <term>
253 |   |                                   |— <factor>
254 |   |                                       |— <procedure/function-call>
255 |   |                                           |— ident(Twice)
256 |   |                                               |— lparent
257 |   |                                                   |— <parameter-list>
258 |   |                                                       |— <expression>
259 |   |                                                           |— <simple-expression>
260 |   |                                                               |— <term>
261 |   |                                                                   |— <factor>
262 |   |                                                                       |— intcon(5)
263 |   |                                                                           |— rparent
264 |   |                                                                               |— rparent
265 |   |— semicolon
266 |   |— <statement>
267 |   |   |— <procedure/function-call>
268 |   |       |— ident(SwapLike)
269 |   |           |— lparent
270 |   |               |— <parameter-list>
271 |   |                   |— <expression>
272 |   |                       |— <simple-expression>
273 |   |                           |— <term>
274 |   |                               |— <factor>
275 |   |                                   |— intcon(1)
276 |   |                   |— comma
277 |   |                       |— <expression>
278 |   |                           |— <simple-expression>
279 |   |                               |— <term>
280 |   |                                   |— <factor>
281 |   |                                       |— intcon(2)
282 |   |                                           |— rparent
283 |   |— semicolon
284 |   |— <statement>
285 |   |— endsy
286 |— period
287

```

Test Case 4: tes ekspresi aritmatika dan boolean yang kompleks. Tes ini memastikan parser mampu menangani prioritas operator (operator precedence) dengan benar (misalnya, perkalian dieksekusi sebelum penjumlahan) serta penggunaan tanda kurung

test > milestone-2 > tc4.txt

```
1  program Expressions;
2  var
3      a, b, c: integer;
4      res: boolean;
5  begin
6      a := 5 + 10 * 2;
7      b := (5 + 10) * 2;
8      c := -a + b div 2;
9      res := (a > b) and not (c == 0) or (a < 100);
10 end.
11
```

```

test > milestone-2 > tc4.txt_OUTPUT.txt
1  <program>
2  |<program-header>
3  |  |<programsy>
4  |  |ident(Expressions)
5  |  |semicolon
6  |<declaration-part>
7  |  |<var-declaration>
8  |  |  |<varsy>
9  |  |  |<identifier-list>
10 |  |  |  |<ident(a)>
11 |  |  |  |comma
12 |  |  |  |<ident(b)>
13 |  |  |  |comma
14 |  |  |  |<ident(c)>
15 |  |  |colon
16 |  |  |<type>
17 |  |  |  |<ident(integer)>
18 |  |  |semicolon
19 |  |  |<identifier-list>
20 |  |  |  |<ident(res)>
21 |  |  |colon
22 |  |  |<type>
23 |  |  |  |<ident(boolean)>
24 |  |  |semicolon
25 |<compound-statement>
26 |  |<beginsy>
27 |  |<statement-list>
28 |  |  |<statement>
29 |  |  |  |<assignment-statement>
30 |  |  |  |  |<variable>
31 |  |  |  |  |  |<ident(a)>
32 |  |  |  |  |becomes
33 |  |  |  |  |<expression>
34 |  |  |  |  |  |<simple-expression>
35 |  |  |  |  |  |  |<term>
36 |  |  |  |  |  |  |  |<factor>
37 |  |  |  |  |  |  |  |  |<intcon(5)>
38 |  |  |  |  |  |  |  |<additive-operator>
39 |  |  |  |  |  |  |  |  |<plus>
40 |  |  |  |  |  |  |  |<term>
41 |  |  |  |  |  |  |  |  |<factor>
42 |  |  |  |  |  |  |  |  |  |<intcon(10)>
43 |  |  |  |  |  |  |  |  |<multiplicative-operator>
44 |  |  |  |  |  |  |  |  |  |<times>
45 |  |  |  |  |  |  |  |  |<factor>
46 |  |  |  |  |  |  |  |  |  |<intcon(2)>
47 |  |  |  |  |  |  |  |semicolon
48 |  |  |  |<statement>
49 |  |  |  |  |<assignment-statement>
50 |  |  |  |  |  |<variable>
51 |  |  |  |  |  |  |<ident(b)>
52 |  |  |  |  |becomes
53 |  |  |  |  |<expression>
54 |  |  |  |  |  |<simple-expression>
55 |  |  |  |  |  |  |<term>
56 |  |  |  |  |  |  |  |<factor>
57 |  |  |  |  |  |  |  |  |<lparent>
58 |  |  |  |  |  |  |  |  |<expression>
59 |  |  |  |  |  |  |  |  |  |<simple-expression>
60 |  |  |  |  |  |  |  |  |  |  |<term>
61 |  |  |  |  |  |  |  |  |  |  |  |<factor>
62 |  |  |  |  |  |  |  |  |  |  |  |  |<intcon(5)>
63 |  |  |  |  |  |  |  |  |  |  |  |<additive-operator>
64 |  |  |  |  |  |  |  |  |  |  |  |  |<plus>
65 |  |  |  |  |  |  |  |  |  |  |  |<term>
66 |  |  |  |  |  |  |  |  |  |  |  |  |<factor>
67 |  |  |  |  |  |  |  |  |  |  |  |  |  |<intcon(10)>
68 |  |  |  |  |  |  |  |  |  |  |  |  |<rparent>
69 |  |  |  |  |  |  |  |  |  |<multiplicative-operator>
70 |  |  |  |  |  |  |  |  |  |  |<times>
71 |  |  |  |  |  |  |  |  |<factor>
72 |  |  |  |  |  |  |  |  |  |<intcon(2)>
73 |  |  |  |  |  |  |  |semicolon

```

```

73 | semicolon
74 | <statement>
75 |   <assignment-statement>
76 |     <variable>
77 |       ident(c)
78 |     becomes
79 |     <expression>
80 |       <simple-expression>
81 |         minus
82 |         <term>
83 |           <factor>
84 |             <variable>
85 |               ident(a)
86 |           <additive-operator>
87 |             plus
88 |           <term>
89 |             <factor>
90 |               <variable>
91 |                 ident(b)
92 |             <multiplicative-operator>
93 |               idiv
94 |             <factor>
95 |               intcon(2)
96 | semicolon
97 | <statement>
98 |   <assignment-statement>
99 |     <variable>
100 |       ident(res)
101 |     becomes
102 |     <expression>
103 |       <simple-expression>
104 |         <term>
105 |           <factor>
106 |             lparent
107 |             <expression>
108 |               <simple-expression>
109 |                 <term>
110 |                   <factor>
111 |                     <variable>
112 |                       ident(a)
113 |                   <relational-operator>
114 |                     gtr
115 |                   <simple-expression>
116 |                     <term>
117 |                       <factor>
118 |                         <variable>
119 |                           ident(b)
120 |             rparent
121 |           <multiplicative-operator>
122 |             andsy
123 |           <factor>
124 |             notsy
125 |           <factor>
126 |             lparent
127 |             <expression>
128 |               <simple-expression>
129 |                 <term>
130 |                   <factor>
131 |                     <variable>
132 |                       ident(c)
133 |                   <relational-operator>
134 |                     eql
135 |                   <simple-expression>
136 |                     <term>
137 |                       <factor>
138 |                         intcon(0)
139 |             rparent
140 |           <additive-operator>
141 |             orsy
142 |           <term>
143 |             <factor>
144 |               lparent
145 |               <expression>
146 |                 <simple-expression>
147 |                   <term>
148 |                     <factor>
149 |                       <variable>
150 |                         ident(a)
151 |                   <relational-operator>
152 |                     lss
153 |                   <simple-expression>
154 |                     <term>
155 |                       <factor>
156 |                         intcon(100)
157 |             rparent
158 | semicolon
159 | <statement>
160 | endsy
161 | period
162 |

```


Test Case 5: tes logika kondisional dan masalah "dangling else" dengan membuat pernyataan if yang saling bersarang (nested) satu sama lain

```
test > milestone-2 > ≡ tc5.txt
1  program IfStatement;
2  var
3  |   x, y: integer;
4  begin
5  |   if x > 0 then
6  |       if y > 0 then
7  |           x := x + 1
8  |       else
9  |           x := x - 1
10 |   else
11 |       x := 0;
12 end.
13
```

```

test > milestone-2 > tc5.txt_OUTPUT.txt
1  <program>
2  |— <program-header>
3  |   |— programsy
4  |   |— ident(IfStatement)
5  |   |— semicolon
6  |— <declaration-part>
7  |   |— <var-declaration>
8  |   |   |— varsy
9  |   |   |— <identifier-list>
10 |   |   |   |— ident(x)
11 |   |   |   |— comma
12 |   |   |   |— ident(y)
13 |   |   |— colon
14 |   |   |— <type>
15 |   |   |   |— ident(integer)
16 |   |   |— semicolon
17 |— <compound-statement>
18 |   |— beginsy
19 |   |— <statement-list>
20 |   |   |— <statement>
21 |   |   |   |— <if-statement>
22 |   |   |   |   |— ifsy
23 |   |   |   |   |— <expression>
24 |   |   |   |   |   |— <simple-expression>
25 |   |   |   |   |   |   |— <term>
26 |   |   |   |   |   |   |   |— <factor>
27 |   |   |   |   |   |   |       |— <variable>
28 |   |   |   |   |   |   |       |   |— ident(x)
29 |   |   |   |   |   |   |— <relational-operator>
30 |   |   |   |   |   |   |   |— gtr
31 |   |   |   |   |   |   |— <simple-expression>
32 |   |   |   |   |   |   |   |— <term>
33 |   |   |   |   |   |   |   |   |— <factor>
34 |   |   |   |   |   |   |   |       |— intcon(0)
35 |   |   |   |— thensy
36 |   |   |   |— <statement>
37 |   |   |   |   |— <if-statement>
38 |   |   |   |   |   |— ifsy
39 |   |   |   |   |   |— <expression>
40 |   |   |   |   |   |   |— <simple-expression>
41 |   |   |   |   |   |   |   |— <term>
42 |   |   |   |   |   |   |   |   |— <factor>
43 |   |   |   |   |   |   |   |       |— <variable>
44 |   |   |   |   |   |   |   |       |   |— ident(y)
45 |   |   |   |   |   |   |— <relational-operator>
46 |   |   |   |   |   |   |   |— gtr
47 |   |   |   |   |   |   |— <simple-expression>
48 |   |   |   |   |   |   |   |— <term>
49 |   |   |   |   |   |   |   |   |— <factor>
50 |   |   |   |   |   |   |   |       |— intcon(0)
51 |   |   |   |— thensy
52 |   |   |   |— <statement>
53 |   |   |   |   |— <assignment-statement>
54 |   |   |   |   |   |— <variable>
55 |   |   |   |   |   |   |— ident(x)
56 |   |   |   |   |   |— becomes
57 |   |   |   |   |   |— <expression>
58 |   |   |   |   |   |   |— <simple-expression>
59 |   |   |   |   |   |   |   |— <term>
60 |   |   |   |   |   |   |   |   |— <factor>
61 |   |   |   |   |   |   |   |       |— <variable>
62 |   |   |   |   |   |   |   |       |   |— ident(x)
63 |   |   |   |   |   |   |— <additive-operator>
64 |   |   |   |   |   |   |   |— plus
65 |   |   |   |   |   |   |— <term>
66 |   |   |   |   |   |   |   |   |— <factor>
67 |   |   |   |   |   |   |   |       |— intcon(1)

```

```

67 |                                     | intcon(1)
68 |                                     |
69 |                                     |
70 |                                     |
71 |                                     |
72 |                                     |
73 |                                     |
74 |                                     |
75 |                                     |
76 |                                     |
77 |                                     |
78 |                                     |
79 |                                     |
80 |                                     |
81 |                                     |
82 |                                     |
83 |                                     |
84 |                                     |
85 |                                     |
86 |                                     |
87 |                                     |
88 |                                     |
89 |                                     |
90 |                                     |
91 |                                     |
92 |                                     |
93 |                                     |
94 |                                     |
95 |                                     |
96 |                                     |
97 |                                     |
98 |                                     |
99 |                                     |
100 |

```

Test Case 6: tes syntax pernyataan case, termasuk penggunaan beberapa konstanta per case (misalnya, 'B', 'C') dan berbagai tipe pernyataan di dalam case tersebut

```

test > milestone-2 > tc6.txt
1  program CaseStatement;
2  var
3  |   grade: char;
4  begin
5  |   case grade of
6  |       'A': writeln('Excellent');
7  |       'B', 'C': writeln('Well done');
8  |       'D': writeln('Passed');
9  |       'F': writeln('Fail');
10 |   end;
11 end.
12

```

```

test > milestone-2 > tc6.txt_OUTPUT.txt
1  <program>
2  |— <program-header>
3  |   |— programsy
4  |   |   |— ident(CaseStatement)
5  |   |   |— semicolon
6  |— <declaration-part>
7  |   |— <var-declaration>
8  |   |   |— varsy
9  |   |   |   |— <identifier-list>
10 |   |   |   |   |— ident(grade)
11 |   |   |   |— colon
12 |   |   |   |— <type>
13 |   |   |   |   |— ident(char)
14 |   |   |   |— semicolon
15 |— <compound-statement>
16 |   |— beginsy
17 |   |   |— <statement-list>
18 |   |   |   |— <statement>
19 |   |   |   |   |— <case-statement>
20 |   |   |   |   |   |— casesy
21 |   |   |   |   |   |   |— <expression>
22 |   |   |   |   |   |   |   |— <simple-expression>
23 |   |   |   |   |   |   |   |   |— <term>
24 |   |   |   |   |   |   |   |   |   |— <factor>
25 |   |   |   |   |   |   |   |   |   |   |— <variable>
26 |   |   |   |   |   |   |   |   |   |   |   |— ident(grade)
27 |   |   |   |   |   |   |   |   |   |— ofsy
28 |   |   |   |   |   |   |   |   |   |   |— <case-block>
29 |   |   |   |   |   |   |   |   |   |   |   |— <constant>
30 |   |   |   |   |   |   |   |   |   |   |   |   |— charcon('A')
31 |   |   |   |   |   |   |   |   |   |   |   |— colon
32 |   |   |   |   |   |   |   |   |   |   |   |   |— <statement>
33 |   |   |   |   |   |   |   |   |   |   |   |   |   |— <procedure/function-call>
34 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— ident(writeln)
35 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— lparent
36 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <parameter-list>
37 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <expression>
38 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <simple-expression>
39 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <term>
40 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <factor>
41 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— string('Excellent')
42 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— rparent
43 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— semicolon
44 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <case-block>
45 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <constant>
46 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— charcon('B')
47 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— comma
48 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <constant>
49 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— charcon('C')
50 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— colon
51 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <statement>
52 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <procedure/function-call>
53 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— ident(writeln)
54 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— lparent
55 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <parameter-list>
56 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <expression>
57 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <simple-expression>
58 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <term>
59 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— <factor>
60 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— string('Well done')
61 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |— rparent

```



Test Case 7: tes struktur perulangan baik untuk pra-kondisi (while) maupun pasca-kondisi (repeat-until)

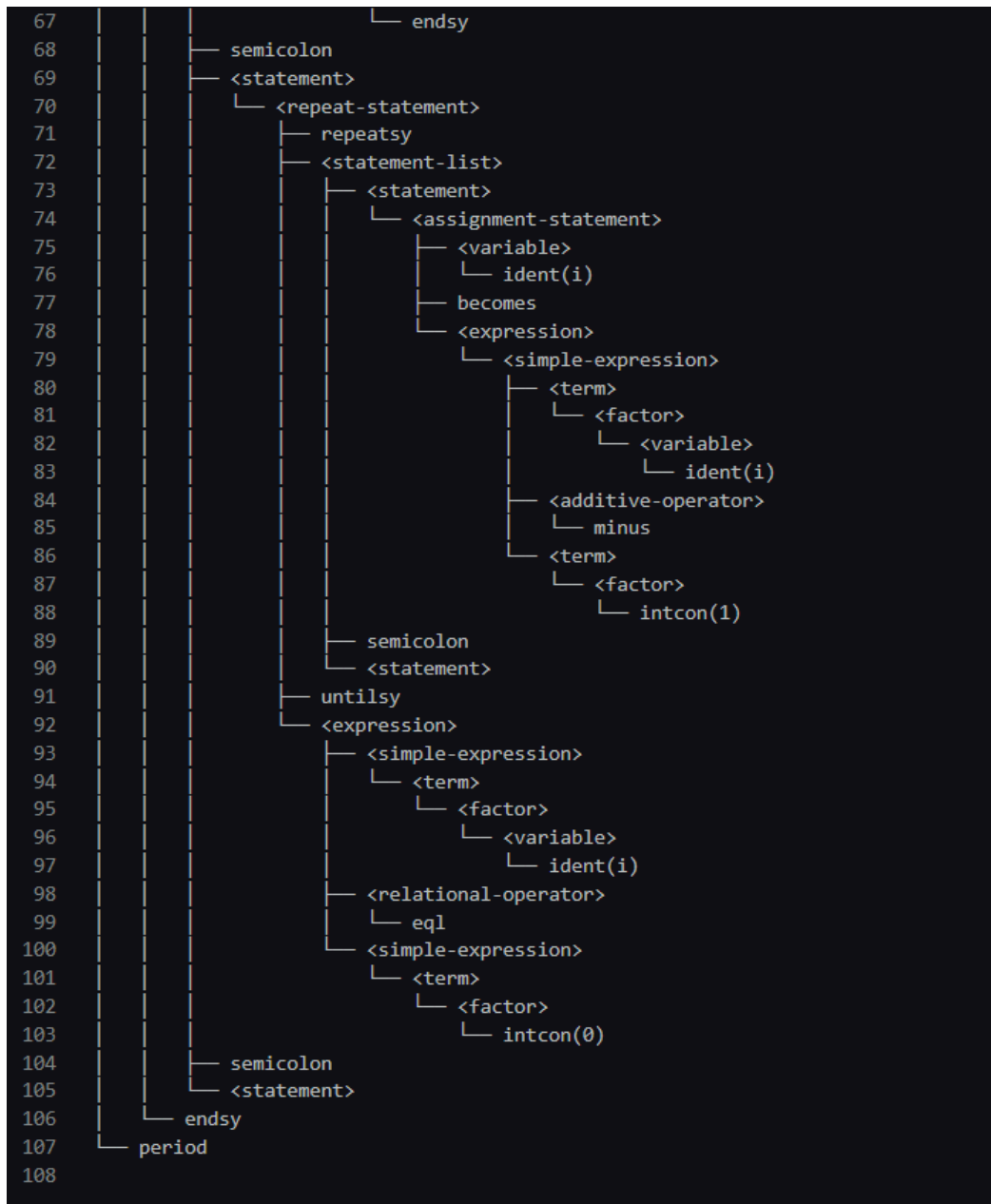
```
test > milestone-2 > tc7.txt
```

```
1  program Loops;  
2  var  
3    i: integer;  
4  begin  
5    i := 0;  
6    while i < 10 do  
7      begin  
8        i := i + 1;  
9      end;  
10   repeat  
11     i := i - 1;  
12   until i == 0;  
13 end.  
14
```

```

test > milestone-2 > tc7.txt_OUTPUT.txt
1  <program>
2  |
3  |   <program-header>
4  |   |
5  |   |   programsy
6  |   |   ident(Loops)
7  |   |   semicolon
8  |   |
9  |   |   <declaration-part>
10 |   |   |
11 |   |   |   <var-declaration>
12 |   |   |   |
13 |   |   |   |   varsy
14 |   |   |   |   |
15 |   |   |   |   |   <identifier-list>
16 |   |   |   |   |   |
17 |   |   |   |   |   |   ident(i)
18 |   |   |   |   |   |   colon
19 |   |   |   |   |   |   <type>
20 |   |   |   |   |   |   |
21 |   |   |   |   |   |   |   ident(integer)
22 |   |   |   |   |   |   |   semicolon
23 |   |   |
24 |   |   |   <compound-statement>
25 |   |   |   |
26 |   |   |   |   beginsy
27 |   |   |   |   |
28 |   |   |   |   |   <statement-list>
29 |   |   |   |   |   |
30 |   |   |   |   |   |   <statement>
31 |   |   |   |   |   |   |
32 |   |   |   |   |   |   |   <assignment-statement>
33 |   |   |   |   |   |   |   |
34 |   |   |   |   |   |   |   |   <variable>
35 |   |   |   |   |   |   |   |   |
36 |   |   |   |   |   |   |   |   |   ident(i)
37 |   |   |   |   |   |   |   |   |   becomes
38 |   |   |   |   |   |   |   |   |   <expression>
39 |   |   |   |   |   |   |   |   |   |
40 |   |   |   |   |   |   |   |   |   |   <simple-expression>
41 |   |   |   |   |   |   |   |   |   |   |
42 |   |   |   |   |   |   |   |   |   |   |   <term>
43 |   |   |   |   |   |   |   |   |   |   |   |
44 |   |   |   |   |   |   |   |   |   |   |   |   <factor>
45 |   |   |   |   |   |   |   |   |   |   |   |   |
46 |   |   |   |   |   |   |   |   |   |   |   |   |   intcon(0)
47 |   |   |   |   |   |   |   |   |   |   |   |   semicolon
48 |   |   |   |   |   |   |   |   |   |   |   |   <statement>
49 |   |   |   |   |   |   |   |   |   |   |   |   |
50 |   |   |   |   |   |   |   |   |   |   |   |   |   <while-statement>
51 |   |   |   |   |   |   |   |   |   |   |   |   |   |
52 |   |   |   |   |   |   |   |   |   |   |   |   |   |   whilesy
53 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
54 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <expression>
55 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
56 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <simple-expression>
57 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
58 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <term>
59 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
60 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <factor>
61 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
62 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <variable>
63 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
64 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   ident(i)
65 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <relational-operator>
66 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
67 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   lss
68 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <simple-expression>
69 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
70 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <term>
71 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
72 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <factor>
73 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
74 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   intcon(10)
75 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   dosy
76 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <statement>
77 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
78 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <compound-statement>
79 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
80 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   beginsy
81 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
82 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <statement-list>
83 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
84 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <statement>
85 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
86 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <assignment-statement>
87 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
88 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <variable>
89 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
90 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   ident(i)
91 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   becomes
92 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <expression>
93 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
94 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <simple-expression>
95 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
96 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <term>
97 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
98 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <factor>
99 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
100|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <variable>
101|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
102|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   ident(i)
103|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <additive-operator>
104|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
105|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   plus
106|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <term>
107|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
108|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <factor>
109|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
110|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   intcon(1)
111|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   semicolon
112|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   <statement>
113|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   endsy

```



Test Case 8: tes tata bahasa (grammar) perulangan for yang menggunakan kata kunci to (inkremen/naik) maupun downto (dekremen/turun)


```
test > milestone-2 > tc8.txt
1  program ForLoops;
2  var
3    i, sum: integer;
4  begin
5    sum := 0;
6    for i := 1 to 10 do
7      sum := sum + i;
8    for i := 10 downto 1 do
9      sum := sum - 1;
10 end.
11
```

```

test > milestone-2 > tc8.txt_OUTPUT.txt
1  <program>
2  |— <program-header>
3  |   |— programsy
4  |   |— ident(ForLoops)
5  |   |— semicolon
6  |— <declaration-part>
7  |   |— <var-declaration>
8  |   |   |— varsy
9  |   |   |— <identifier-list>
10 |   |   |   |— ident(i)
11 |   |   |   |— comma
12 |   |   |   |— ident(sum)
13 |   |   |— colon
14 |   |   |— <type>
15 |   |   |   |— ident(integer)
16 |   |   |— semicolon
17 |— <compound-statement>
18 |   |— beginsy
19 |   |— <statement-list>
20 |   |   |— <statement>
21 |   |   |   |— <assignment-statement>
22 |   |   |   |   |— <variable>
23 |   |   |   |   |   |— ident(sum)
24 |   |   |   |   |— becomes
25 |   |   |   |   |— <expression>
26 |   |   |   |   |   |— <simple-expression>
27 |   |   |   |   |   |   |— <term>
28 |   |   |   |   |   |   |   |— <factor>
29 |   |   |   |   |   |   |   |— intcon(0)
30 |   |   |— semicolon
31 |   |   |— <statement>
32 |   |   |   |— <for-statement>
33 |   |   |   |   |— forsy
34 |   |   |   |   |— ident(i)
35 |   |   |   |   |— becomes
36 |   |   |   |   |— <expression>
37 |   |   |   |   |   |— <simple-expression>
38 |   |   |   |   |   |   |— <term>
39 |   |   |   |   |   |   |   |— <factor>
40 |   |   |   |   |   |   |   |— intcon(1)
41 |   |   |   |   |— tosy
42 |   |   |   |   |— <expression>
43 |   |   |   |   |   |— <simple-expression>
44 |   |   |   |   |   |   |— <term>
45 |   |   |   |   |   |   |   |— <factor>
46 |   |   |   |   |   |   |   |— intcon(10)
47 |   |   |   |— dosy
48 |   |   |   |— <statement>
49 |   |   |   |   |— <assignment-statement>
50 |   |   |   |   |   |— <variable>
51 |   |   |   |   |   |   |— ident(sum)
52 |   |   |   |   |   |— becomes
53 |   |   |   |   |   |— <expression>
54 |   |   |   |   |   |   |— <simple-expression>
55 |   |   |   |   |   |   |   |— <term>
56 |   |   |   |   |   |   |   |   |— <factor>
57 |   |   |   |   |   |   |   |   |   |— <variable>
58 |   |   |   |   |   |   |   |   |   |   |— ident(sum)
59 |   |   |   |   |   |   |   |— <additive-operator>
60 |   |   |   |   |   |   |   |   |— plus
61 |   |   |   |   |   |   |   |— <term>
62 |   |   |   |   |   |   |   |   |— <factor>
63 |   |   |   |   |   |   |   |   |   |— <variable>
64 |   |   |   |   |   |   |   |   |   |   |— ident(i)

```

```

64                                     └─ ident(i)
65 ── semicolon
66 ── <statement>
67   └─ <for-statement>
68     └─ forsy
69       └─ ident(i)
70       └─ becomes
71       └─ <expression>
72         └─ <simple-expression>
73           └─ <term>
74             └─ <factor>
75               └─ intcon(10)
76       └─ downtosy
77       └─ <expression>
78         └─ <simple-expression>
79           └─ <term>
80             └─ <factor>
81               └─ intcon(1)
82       └─ dosy
83       └─ <statement>
84         └─ <assignment-statement>
85           └─ <variable>
86             └─ ident(sum)
87           └─ becomes
88           └─ <expression>
89             └─ <simple-expression>
90               └─ <term>
91                 └─ <factor>
92                   └─ <variable>
93                     └─ ident(sum)
94               └─ <additive-operator>
95                 └─ minus
96               └─ <term>
97                 └─ <factor>
98                   └─ intcon(1)
99   └─ semicolon
100 └─ <statement>
101 └─ endsy
102 └─ period
103

```

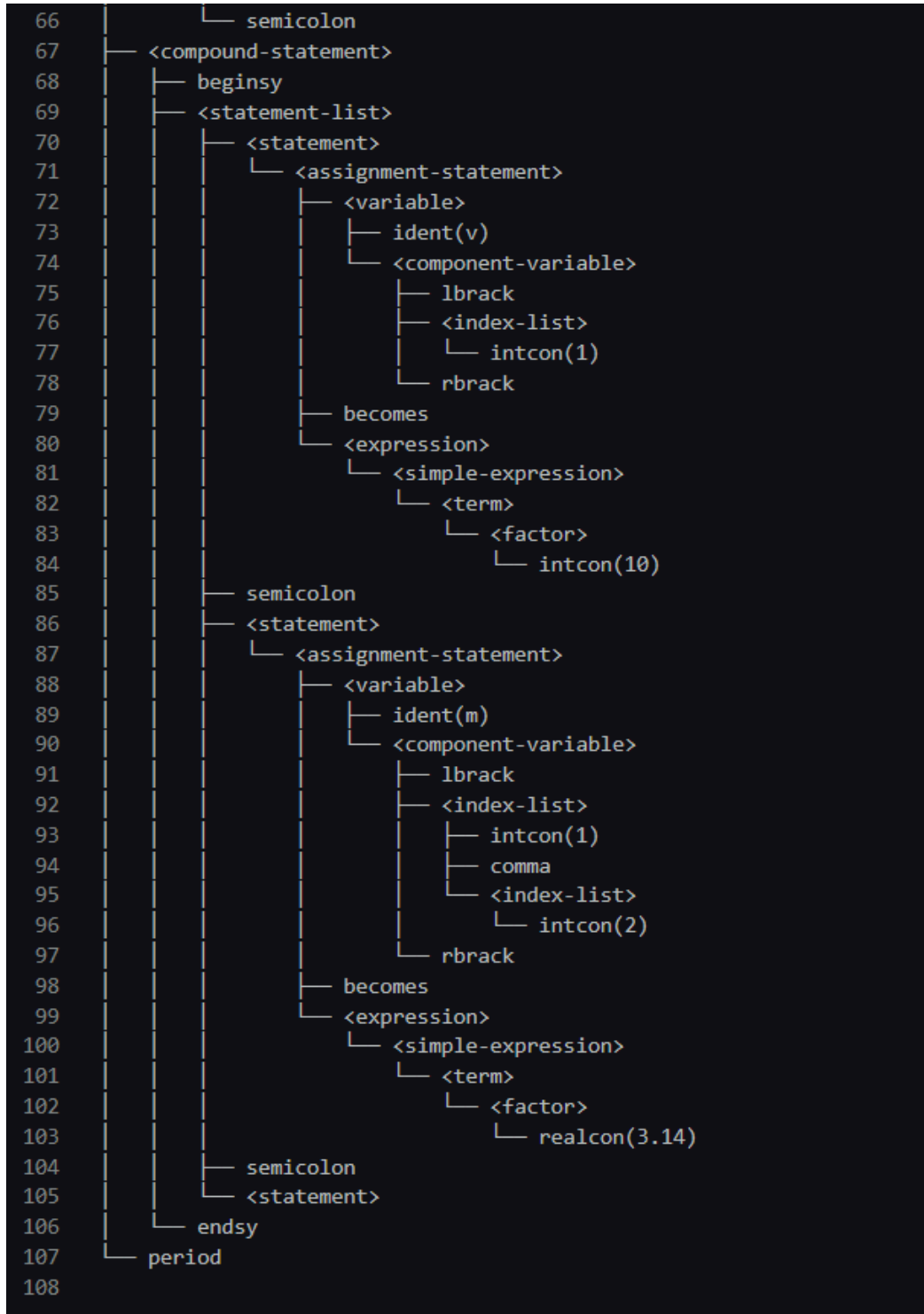
Test Case 9: tes deklarasi array multi-dimensi dan sintaksis untuk pengindeksan array (v[1], m[1, 2])

```
test > milestone-2 > ≡ tc9.txt
1  program Arrays;
2  type
3  |   Vec == array[1..5] of integer;
4  var
5  |   v: Vec;
6  |   m: array[1..3] of array[1..3] of real;
7  begin
8  |   v[1] := 10;
9  |   m[1, 2] := 3.14;
10 end.
11
```

```

test > milestone-2 > tc9.txt_OUTPUT.txt
1  <program>
2  |— <program-header>
3  |   |— programsy
4  |   |— ident(Arrays)
5  |   |— semicolon
6  |— <declaration-part>
7  |   |— <type-declaration>
8  |   |   |— typesy
9  |   |   |— ident(Vec)
10 |   |   |— eql
11 |   |   |— <type>
12 |   |   |   |— <array-type>
13 |   |   |   |   |— arraysy
14 |   |   |   |   |— lbrack
15 |   |   |   |   |— <range>
16 |   |   |   |   |   |— <constant>
17 |   |   |   |   |   |   |— intcon(1)
18 |   |   |   |   |   |   |— period
19 |   |   |   |   |   |   |— period
20 |   |   |   |   |   |   |— <constant>
21 |   |   |   |   |   |   |— intcon(5)
22 |   |   |   |   |— rbrack
23 |   |   |   |   |— ofsy
24 |   |   |   |   |— <type>
25 |   |   |   |   |   |— ident(integer)
26 |   |   |— semicolon
27 |   |— <var-declaration>
28 |   |   |— varsy
29 |   |   |— <identifier-list>
30 |   |   |   |— ident(v)
31 |   |   |— colon
32 |   |   |— <type>
33 |   |   |   |— ident(Vec)
34 |   |   |— semicolon
35 |   |   |— <identifier-list>
36 |   |   |   |— ident(m)
37 |   |   |— colon
38 |   |   |— <type>
39 |   |   |   |— <array-type>
40 |   |   |   |   |— arraysy
41 |   |   |   |   |— lbrack
42 |   |   |   |   |— <range>
43 |   |   |   |   |   |— <constant>
44 |   |   |   |   |   |   |— intcon(1)
45 |   |   |   |   |   |   |— period
46 |   |   |   |   |   |   |— period
47 |   |   |   |   |   |   |— <constant>
48 |   |   |   |   |   |   |— intcon(3)
49 |   |   |   |   |— rbrack
50 |   |   |   |   |— ofsy
51 |   |   |   |   |— <type>
52 |   |   |   |   |   |— <array-type>
53 |   |   |   |   |   |   |— arraysy
54 |   |   |   |   |   |   |— lbrack
55 |   |   |   |   |   |   |— <range>
56 |   |   |   |   |   |   |   |— <constant>
57 |   |   |   |   |   |   |   |   |— intcon(1)
58 |   |   |   |   |   |   |   |   |— period
59 |   |   |   |   |   |   |   |   |— period
60 |   |   |   |   |   |   |   |   |— <constant>
61 |   |   |   |   |   |   |   |   |— intcon(3)
62 |   |   |   |   |— rbrack
63 |   |   |   |   |— ofsy
64 |   |   |   |   |— <type>
65 |   |   |   |   |   |— ident(real)
66 |   |   |— semicolon

```



Test Case 10: tes definisi tipe record dan penggunaan operator titik untuk mengakses field (bidang), termasuk mengakses field di dalam sebuah array of records

```
test > milestone-2 > tc10.txt
1  program Records;
2  type
3      Point == record
4          |   x, y: integer;
5          |   end;
6  var
7      p: Point;
8      points: array[1..10] of Point;
9  begin
10     p.x := 10;
11     p.y := 20;
12     points[1].x := p.x;
13 end.
14
```

```

test > milestone-2 > tc10.txt_OUTPUT.txt
1  <program>
2  |— <program-header>
3  |   |— programsy
4  |   |— ident(Records)
5  |   |— semicolon
6  |— <declaration-part>
7  |   |— <type-declaration>
8  |   |   |— typesy
9  |   |   |— ident(Point)
10 |   |   |— eql
11 |   |   |— <type>
12 |   |       |— <record-type>
13 |   |           |— recordsy
14 |   |           |— <field-list>
15 |   |               |— <field-part>
16 |   |                   |— <identifier-list>
17 |   |                       |— ident(x)
18 |   |                           |— comma
19 |   |                               |— ident(y)
20 |   |                                   |— colon
21 |   |                                       |— <type>
22 |   |                                           |— ident(integer)
23 |   |                                               |— semicolon
24 |   |   |— endsy
25 |   |— semicolon
26 |— <var-declaration>
27 |   |— varsy
28 |   |— <identifier-list>
29 |   |   |— ident(p)
30 |   |   |— colon
31 |   |   |— <type>
32 |   |       |— ident(Point)
33 |   |       |— semicolon
34 |   |       |— <identifier-list>
35 |   |           |— ident(points)
36 |   |           |— colon
37 |   |           |— <type>
38 |   |               |— <array-type>
39 |   |                   |— arraysy
40 |   |                   |— lbrack
41 |   |                   |— <range>
42 |   |                       |— <constant>
43 |   |                           |— intcon(1)
44 |   |                           |— period
45 |   |                           |— period
46 |   |                           |— <constant>
47 |   |                               |— intcon(10)
48 |   |                   |— rbrack
49 |   |                   |— ofsy
50 |   |                   |— <type>
51 |   |                       |— ident(Point)
52 |   |— semicolon

```



```

52 |         | semicolon
53 |     | <compound-statement>
54 |     | | beginsy
55 |     | | <statement-list>
56 |     | | | <statement>
57 |     | | | | <assignment-statement>
58 |     | | | | | <variable>
59 |     | | | | | | ident(p)
60 |     | | | | | | <component-variable>
61 |     | | | | | | | period
62 |     | | | | | | | ident(x)
63 |     | | | | | becomes
64 |     | | | | | <expression>
65 |     | | | | | | <simple-expression>
66 |     | | | | | | | <term>
67 |     | | | | | | | | <factor>
68 |     | | | | | | | | | intcon(10)
69 |     | | semicolon
70 |     | | <statement>
71 |     | | | <assignment-statement>
72 |     | | | | <variable>
73 |     | | | | | ident(p)
74 |     | | | | | <component-variable>
75 |     | | | | | | period
76 |     | | | | | | ident(y)
77 |     | | | | | becomes
78 |     | | | | | <expression>
79 |     | | | | | | <simple-expression>
80 |     | | | | | | | <term>
81 |     | | | | | | | | <factor>
82 |     | | | | | | | | | intcon(20)
83 |     | | semicolon
84 |     | | <statement>
85 |     | | | <assignment-statement>
86 |     | | | | <variable>
87 |     | | | | | ident(points)
88 |     | | | | | <component-variable>
89 |     | | | | | | lbrack
90 |     | | | | | | | <index-list>
91 |     | | | | | | | | intcon(1)
92 |     | | | | | | | rbrack
93 |     | | | | | <component-variable>
94 |     | | | | | | period
95 |     | | | | | | ident(x)
96 |     | | | | | becomes
97 |     | | | | | <expression>
98 |     | | | | | | <simple-expression>
99 |     | | | | | | | <term>
100 |     | | | | | | | | <factor>
101 |     | | | | | | | | | <variable>
102 |     | | | | | | | | | | ident(p)
103 |     | | | | | | | | | | <component-variable>
104 |     | | | | | | | | | | | period
105 |     | | | | | | | | | | | ident(x)
106 |     | | semicolon
107 |     | | <statement>
108 |     | endsy
109 | | period
110 |

```

Test Case 11: tes pemanggilan fungsi (Faktorial) yang digunakan sebagai faktor di dalam ekspresinya sendiri untuk memastikan bahwa pemanggilan rekursif berfungsi dengan baik di dalam tata bahasa tersebut

```
test > milestone-2 > ≡ tc11.txt
1  program Calls;
2  function Factorial(n: integer): integer;
3  ∨ begin
4  |   if n == 0 then Factorial := 1
5  |   else Factorial := n * Factorial(n - 1);
6  |   end;
7  ∨ begin
8  |   writeln(Factorial(5));
9  |   end.
10
```

[illegible]



Test Case 12: tes ketahanan (stress test) yang menggabungkan record, array, perulangan bersarang (nested loops), dan pernyataan kondisional untuk memastikan parser mampu menangani hierarki struktur yang dalam dan kompleks

test > milestone-2 > tc12.txt

```
1  program Complex;
2  type
3  |   ScoreIndex == 1..5;
4  |   Student == record
5  |     id: integer;
6  |     grades: array[ScoreIndex] of integer;
7  |   end;
8  |   Classroom == array[1..3] of Student;
9  var
10 |   classes: Classroom;
11 |   i, j, total: integer;
12 begin
13 |   total := 0;
14 |   for i := 1 to 3 do
15 |     begin
16 |       classes[i].id := i;
17 |       j := 1;
18 |       while j <= 5 do
19 |         begin
20 |           if classes[i].grades[j] >= 60 then
21 |             total := total + classes[i].grades[j]
22 |           else
23 |             total := total + 0;
24 |             j := j + 1;
25 |           end;
26 |         end;
27 |       repeat
28 |         total := total div 2;
29 |       until total < 100;
30 end.
31
```

```

1  <program>
2  |— <program-header>
3  |   |— programsy
4  |   |— ident(Complex)
5  |   |— semicolon
6  |— <declaration-part>
7  |   |— <type-declaration>
8  |   |   |— typesy
9  |   |   |— ident(ScoreIndex)
10 |   |   |— eql
11 |   |   |— <type>
12 |   |   |   |— <range>
13 |   |   |   |   |— <constant>
14 |   |   |   |   |   |— intcon(1)
15 |   |   |   |   |   |— period
16 |   |   |   |   |   |— period
17 |   |   |   |   |   |— <constant>
18 |   |   |   |   |   |   |— intcon(5)
19 |   |   |— semicolon
20 |   |   |— ident(Student)
21 |   |   |— eql
22 |   |   |— <type>
23 |   |   |   |— <record-type>
24 |   |   |   |   |— recordsy
25 |   |   |   |   |— <field-list>
26 |   |   |   |   |   |— <field-part>
27 |   |   |   |   |   |   |— <identifier-list>
28 |   |   |   |   |   |   |   |— ident(id)
29 |   |   |   |   |   |   |   |— colon
30 |   |   |   |   |   |   |   |— <type>
31 |   |   |   |   |   |   |   |   |— ident(integer)
32 |   |   |   |   |   |— semicolon
33 |   |   |   |   |   |— <field-part>
34 |   |   |   |   |   |   |— <identifier-list>
35 |   |   |   |   |   |   |   |— ident(grades)
36 |   |   |   |   |   |   |   |— colon
37 |   |   |   |   |   |   |   |— <type>
38 |   |   |   |   |   |   |   |   |— <array-type>
39 |   |   |   |   |   |   |   |   |   |— arraysy
40 |   |   |   |   |   |   |   |   |   |— lbrack
41 |   |   |   |   |   |   |   |   |   |— ident(ScoreIndex)
42 |   |   |   |   |   |   |   |   |   |— rbrack
43 |   |   |   |   |   |   |   |   |   |— ofsy
44 |   |   |   |   |   |   |   |   |   |— <type>
45 |   |   |   |   |   |   |   |   |   |   |— ident(integer)
46 |   |   |   |   |   |— semicolon
47 |   |   |   |— endsy
48 |   |— semicolon
49 |   |— ident(Classroom)
50 |   |— eql
51 |   |— <type>
52 |   |   |— <array-type>
53 |   |   |   |— arraysy
54 |   |   |   |— lbrack
55 |   |   |   |— <range>
56 |   |   |   |   |— <constant>
57 |   |   |   |   |   |— intcon(1)
58 |   |   |   |   |   |— period
59 |   |   |   |   |   |— period
60 |   |   |   |   |   |— <constant>
61 |   |   |   |   |   |   |— intcon(3)
62 |   |   |   |   |— rbrack
63 |   |   |   |— ofsy
64 |   |   |   |— <type>
65 |   |   |   |   |— ident(Student)
66 |   |— semicolon
67 |— <var-declaration>

```

```

67      └─ <var-declaration>
68      │   └─ varsy
69      │   │   └─ <identifier-list>
70      │   │   │   └─ ident(classes)
71      │   │   └─ colon
72      │   │   └─ <type>
73      │   │   │   └─ ident(Classroom)
74      │   │   └─ semicolon
75      │   │   └─ <identifier-list>
76      │   │   │   └─ ident(i)
77      │   │   │   └─ comma
78      │   │   │   └─ ident(j)
79      │   │   │   └─ comma
80      │   │   │   └─ ident(total)
81      │   │   └─ colon
82      │   │   └─ <type>
83      │   │   │   └─ ident(integer)
84      │   │   └─ semicolon
85  └─ <compound-statement>
86      │   └─ beginsy
87      │   │   └─ <statement-list>
88      │   │   │   └─ <statement>
89      │   │   │   │   └─ <assignment-statement>
90      │   │   │   │   │   └─ <variable>
91      │   │   │   │   │   │   └─ ident(total)
92      │   │   │   │   │   └─ becomes
93      │   │   │   │   │   └─ <expression>
94      │   │   │   │   │   │   └─ <simple-expression>
95      │   │   │   │   │   │   │   └─ <term>
96      │   │   │   │   │   │   │   │   └─ <factor>
97      │   │   │   │   │   │   │   │   │   └─ intcon(0)
98      │   │   │   │   └─ semicolon
99      │   │   │   └─ <statement>
100      │   │   │   │   └─ <for-statement>
101      │   │   │   │   │   └─ forsy
102      │   │   │   │   │   └─ ident(i)
103      │   │   │   │   │   └─ becomes
104      │   │   │   │   │   └─ <expression>
105      │   │   │   │   │   │   └─ <simple-expression>
106      │   │   │   │   │   │   │   └─ <term>
107      │   │   │   │   │   │   │   │   └─ <factor>
108      │   │   │   │   │   │   │   │   │   └─ intcon(1)
109      │   │   │   │   │   └─ tosy
110      │   │   │   │   │   └─ <expression>
111      │   │   │   │   │   │   └─ <simple-expression>
112      │   │   │   │   │   │   │   └─ <term>
113      │   │   │   │   │   │   │   │   └─ <factor>
114      │   │   │   │   │   │   │   │   │   └─ intcon(3)
115      │   │   │   │   └─ dosy
116      │   │   │   │   └─ <statement>
117      │   │   │   │   │   └─ <compound-statement>
118      │   │   │   │   │   │   └─ beginsy
119      │   │   │   │   │   │   │   └─ <statement-list>
120      │   │   │   │   │   │   │   │   └─ <statement>
121      │   │   │   │   │   │   │   │   │   └─ <assignment-statement>
122      │   │   │   │   │   │   │   │   │   └─ <variable>
123      │   │   │   │   │   │   │   │   │   │   └─ ident(classes)
124      │   │   │   │   │   │   │   │   │   └─ <component-variable>
125      │   │   │   │   │   │   │   │   │   │   └─ lbrack
126      │   │   │   │   │   │   │   │   │   │   └─ <index-list>
127      │   │   │   │   │   │   │   │   │   │   │   └─ ident(i)
128      │   │   │   │   │   │   │   │   │   └─ rbrack
129      │   │   │   │   │   │   │   │   └─ <component-variable>
130      │   │   │   │   │   │   │   │   └─ period
131      │   │   │   │   │   │   │   │   └─ ident(id)

```

```

131      └─ ident(id)
132      └─ becomes
133      └─ <expression>
134          └─ <simple-expression>
135              └─ <term>
136                  └─ <factor>
137                      └─ <variable>
138                          └─ ident(i)
139      └─ semicolon
140      └─ <statement>
141          └─ <assignment-statement>
142              └─ <variable>
143                  └─ ident(j)
144              └─ becomes
145              └─ <expression>
146                  └─ <simple-expression>
147                      └─ <term>
148                          └─ <factor>
149                              └─ intcon(1)
150      └─ semicolon
151      └─ <statement>
152          └─ <while-statement>
153              └─ whilesy
154              └─ <expression>
155                  └─ <simple-expression>
156                      └─ <term>
157                          └─ <factor>
158                              └─ <variable>
159                                  └─ ident(j)
160                  └─ <relational-operator>
161                      └─ leq
162                  └─ <simple-expression>
163                      └─ <term>
164                          └─ <factor>
165                              └─ intcon(5)
166      └─ dosy
167      └─ <statement>
168          └─ <compound-statement>
169              └─ beginsy
170              └─ <statement-list>
171                  └─ <statement>
172                      └─ <if-statement>
173                          └─ ifsy
174                          └─ <expression>
175                              └─ <simple-expression>
176                                  └─ <term>
177                                      └─ <factor>
178                                          └─ <variable>
179                                              └─ ident(classes)
180                                              └─ <component-variable>
181                                                  └─ lbrack
182                                                  └─ <index-list>
183                                                      └─ ident(i)
184                                                  └─ rbrack
185                                              └─ <component-variable>
186                                                  └─ period
187                                                  └─ ident(grades)
188                                              └─ <component-variable>
189                                                  └─ lbrack
190                                                  └─ <index-list>
191                                                      └─ ident(j)
192                                                  └─ rbrack
193                          └─ <relational-operator>
194                              └─ geq
195                          └─ <simple-expression>
196                              └─ <term>
197                                  └─ <factor>
198                                      └─ intcon(60)

```



```

247 | | | | | semicolon
248 | | | | | <statement>
249 | | | | |   <assignment-statement>
250 | | | | |     <variable>
251 | | | | |       ident(j)
252 | | | | |     becomes
253 | | | | |     <expression>
254 | | | | |       <simple-expression>
255 | | | | |         <term>
256 | | | | |           <factor>
257 | | | | |             <variable>
258 | | | | |               ident(j)
259 | | | | |           <additive-operator>
260 | | | | |             plus
261 | | | | |           <term>
262 | | | | |             <factor>
263 | | | | |               intcon(1)
264 | | | | | semicolon
265 | | | | | <statement>
266 | | | | | endsy
267 | | | | | semicolon
268 | | | | | <statement>
269 | | | | | endsy
270 | | | semicolon
271 | | | <statement>
272 | | |   <repeat-statement>
273 | | |     repeatsy
274 | | |       <statement-list>
275 | | |         <statement>
276 | | |           <assignment-statement>
277 | | |             <variable>
278 | | |               ident(total)
279 | | |             becomes
280 | | |             <expression>
281 | | |               <simple-expression>
282 | | |                 <term>
283 | | |                   <factor>
284 | | |                     <variable>
285 | | |                       ident(total)
286 | | |                   <multiplicative-operator>
287 | | |                     idiv
288 | | |                   <factor>
289 | | |                     intcon(2)
290 | | | semicolon
291 | | | <statement>
292 | | |   untilsy
293 | | |     <expression>
294 | | |       <simple-expression>
295 | | |         <term>
296 | | |           <factor>
297 | | |             <variable>
298 | | |               ident(total)
299 | | |           <relational-operator>
300 | | |             lss
301 | | |           <simple-expression>
302 | | |             <term>
303 | | |               <factor>
304 | | |                 intcon(100)
305 | | | semicolon
306 | | | <statement>
307 | | | endsy
308 | | period
309 |

```

IV. Kesimpulan dan Saran

Pada Milestone 2, Arion Compiler telah diperluas dari lexical analyzer menjadi syntax analyzer. Parser menerima token dari lexer, memeriksa kesesuaiannya dengan grammar, lalu membentuk parse tree. Implementasi menggunakan recursive descent parser karena setiap rule grammar dapat dipetakan langsung menjadi fungsi parser.

Parser juga menangani beberapa klarifikasi penting, seperti urutan deklarasi yang wajib, statement kosong, procedure/function call yang harus memakai parameter dalam tanda kurung, enumerated type dengan tanda kurung, variable chain untuk array dan record, serta wrapper node pada operator dan statement.

Untuk pelaksanaan tugas besar di masa mendatang, disarankan agar spesifikasi awal yang diberikan kepada mahasiswa dapat disusun secara lebih detail dan mendalam guna meminimalisir munculnya ambiguitas pada kasus-kasus kompleks. Penjelasan spesifikasi yang lebih rinci sejak awal rilis milestone diharapkan dapat mengurangi frekuensi revisi aturan atau tambahan instruksi, sehingga setiap kelompok memiliki panduan teknis yang lebih pasti, konsisten, dan komprehensif.

V. Lampiran

A. Link Repository Github

<https://github.com/alfarabi0642/PRX-Tubes-IF2224-2026>

B. Pembagian Tugas

NIM	Tugas
13524002	Subprogram declaration, formal parameter, statement, variable chain, assignment, if
13524022	Case, while, repeat, for, procedure/function call, expression, operator precedence
13524086	Parser infrastructure, top-level grammar, parse tree, main.cpp, Makefile
13524094	Constant, type, var, array, range, enumerated, record

VI. Referensi

1. Tutorialspoint. "Compiler Design - Lexical Analysis."
https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.html
2. J. E. Hopcroft, R. Motwani, dan J. D. Ullman. Introduction to Automata Theory, Languages, and Computation, 3rd ed.
https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf

3. A. V. Aho, M. S. Lam, R. Sethi, dan J. D. Ullman. Compilers: Principles, Techniques, and Tools, 2nd ed. (Dragon Book), 2006.
[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20(2006).pdf)